22AIE450 — Speech Processing Lab 3

Name: Gunda Harshavardhan
Roll Number: CB.SC.U4AIE23226

Question 1

Record the utterance "My name is [your name]" in a continuous manner with a pause after each word at a sampling frequency of 16 kHz and bit-resolution of 16, and save the recording in raw WAV format. Compute the LP residual for a 25 ms voiced frame properly selected from the utterance. Comment on the nature and significance of the LP residual plot. Estimate the formant frequencies from the LP spectrum. Write the inference.

Aim

To analyze a voiced speech frame using Linear Predictive Coding (LPC), obtain its LP residual, study the excitation characteristics of the residual, estimate formant frequencies from the LPC/LP spectrum, and interpret the results.

Speech Recording and Parameters

Parameter	Value
Utterance	"My name is Harsha vardhan"
Sampling frequency	16 kHz
Bit resolution	16 bit
File format	WAV
Speech type for analysis	Voiced vowel region

The speech was recorded and converted to a 16-kHz, 16-bit WAV file. The words were spoken continuously, with a short pause between words as required.

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import wavfile
from scipy.signal import lfilter, freqz

# Load 16 kHz, 16-bit WAV
fs, x = wavfile.read("my_name_1.wav")
x = x[:, 0] if x.ndim > 1 else x
x = x.astype(float)
x /= np.max(np.abs(x))

# Select 25 ms voiced frame starting at 2 s
start = int(2 * fs)
N = int(0.025 * fs)
frame = x[start:start + N]

# LPC analysis
frame_w = frame * np.hamming(N)
r = np.correlate(frame_w, frame_w, mode="full")
r = r[N-1:N+13]

R = np.array([[r[abs(i-j)] for j in range(12)] for i in range(12)])
a = np.r_[1, -np.linalg.solve(R, r[1:13])]

# LP residual
residual = lfilter(a, [1], frame_w)

# LP spectrum
f, H = freqz([1], a, worN=4096, fs=fs)
S = 20 * np.log10(np.maximum(abs(H), 1e-10))
```

```
# Formant frequencies from LPC poles
roots = np.roots(a)
roots = roots[np.imag(roots) > 0]
formants = np.sort(np.angle(roots) * fs / (2 * np.pi))
formants = formants[(formants > 200) & (formants < 5000)][:3]

print("Formants:", [f"{v:.2f} Hz" for v in formants])

# Plots
t = np.arange(N) / fs * 1000

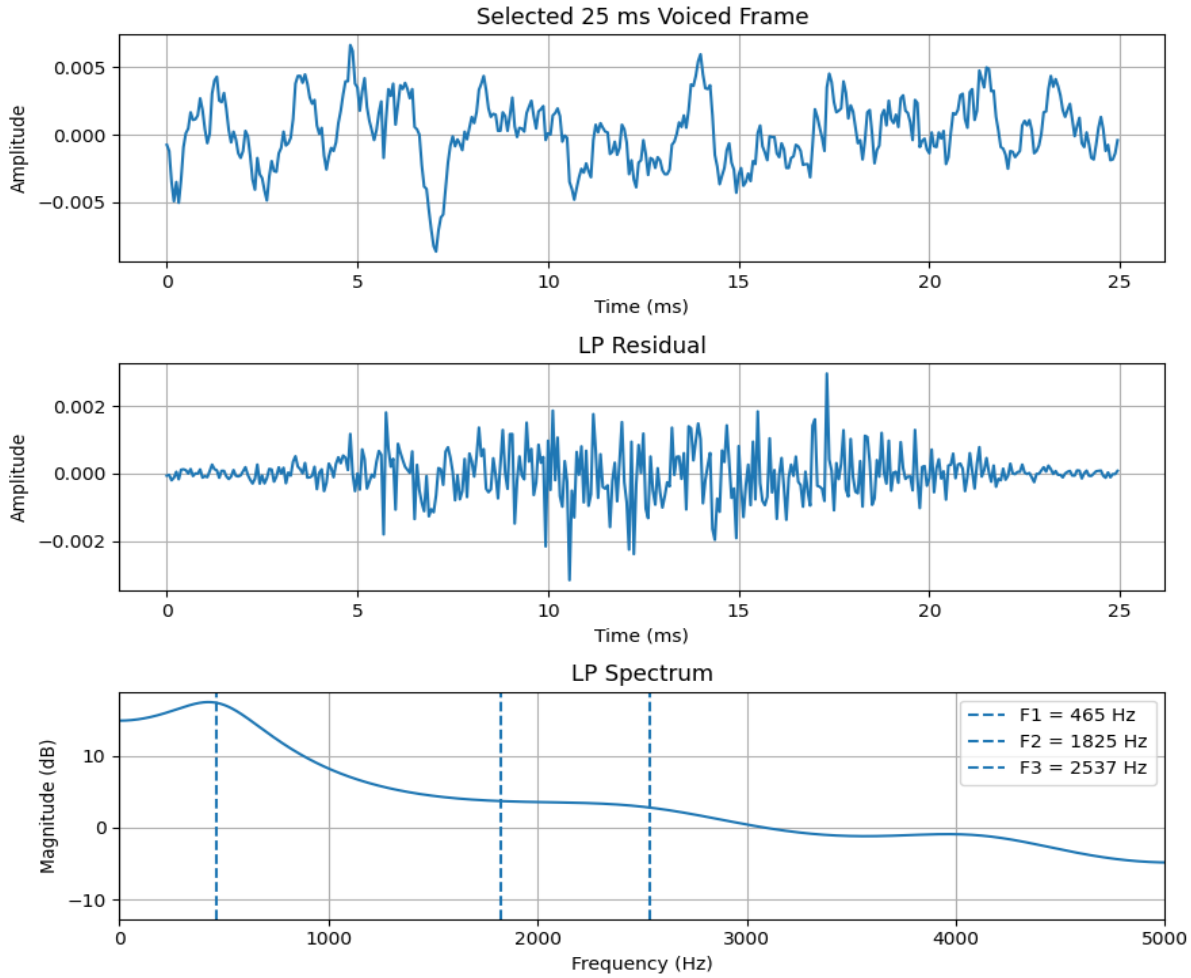
plt.figure(figsize=(9, 8))

plt.subplot(3, 1, 1)
plt.plot(t, frame)
plt.title("Selected 25 ms Voiced Frame")
plt.xlabel("Time (ms)")
plt.ylabel("Amplitude")
plt.grid()

plt.subplot(3, 1, 2)
plt.plot(t, residual)
plt.title("LP Residual")
plt.xlabel("Time (ms)")
plt.ylabel("Amplitude")
plt.grid()

plt.subplot(3, 1, 3)
plt.plot(f, S)
for i, F in enumerate(formants, 1):
    plt.axvline(F, linestyle="--", label=f"F{i} = {F:.0f} Hz")
plt.xlim(0, 5000)
plt.title("LP Spectrum")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude (dB)")
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()
```

Output:



Inference

The selected 25 ms frame represents a voiced portion of the speech signal. The LP residual shows the excitation component after removing the vocal-tract characteristics. The residual contains several prominent peaks and a recurring structure, although it is not perfectly periodic, which can occur due to the short frame duration and characteristics of the recorded speech.

The LP spectrum indicates the resonant characteristics of the vocal tract. The estimated formant frequencies are **F1 = 465 Hz, F2 = 1825 Hz, and F3 = 2537 Hz**. The first formant is clearly associated with the major low-frequency spectral peak, while the higher formants are obtained from the LPC pole analysis and are less visually prominent in this particular frame.

Overall, the LPC analysis successfully separates the **excitation information in the LP residual** from the **vocal-tract resonance information represented by the formants**.

Question 2:

Perform age classification (teens, twenties, thirties, forties) on the 'common voice' Hindi dataset using different Machine learning and deep learning models. Perform all experiments in a speaker-independent manner. The dataset can be downloaded from the following link. https://mozilladatacollective.com/datasets/cmqiod71900zgnr07uiyw57br

Specifically, perform the following:

a. Comparison of features: Compare the performance of the 62 feature-set, 88 features-set from open smile and 180 selected features (which includes MFCC -40 features, Mel spectrogram – 128 features, and Chroma -12 features as shown in Tutorial).

b. 2D CNN on Mel-spectrogram Image: Can a 2D CNN achieve the highest accuracy by directly classifying age based on the image representation of the Mel-spectrogram itself? (when saving image remove the color-bar and axis)

```
import os, sys, json, time, random, warnings, itertools
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import StratifiedGroupKFold
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             classification_report, confusion_matrix, ConfusionMatrixDisplay)
from sklearn.utils.class_weight import compute_class_weight

warnings.filterwarnings("ignore")

try:
    import librosa
    import librosa.display
    HAVE_LIBROSA = True
except ImportError:
    HAVE_LIBROSA = False
    print("librosa not installed -> run: pip install librosa")
try:
    import opensmile
    HAVE_OPENSMILE = True
except ImportError:
    HAVE_OPENSMILE = False
    print("opensmile not installed -> run: pip install opensmile")
try:
    import xgboost as xgb
    HAVE_XGBOOST = True
except ImportError:
    HAVE_XGBOOST = False
    print("xgboost not installed -> falling back to HistGradientBoostingClassifier (reported explicitly wherever
```

```
try:
    import torch
    import torch.nn as nn
    from torch.utils.data import Dataset, DataLoader
    HAVE_TORCH = True
except ImportError:
    HAVE_TORCH = False
    print("torch not installed -> run: pip install torch torchvision")

from tqdm.auto import tqdm

print("Imports complete.")
```

```python
DATASET_ROOT = r"1781715680033-cv-corpus-26.0-2026-06-12-hi/cv-corpus-26.0-2026-06-12/hi"

# If DATASET_ROOT does not resolve automatically, set this manually, e.g.
# MANUAL_TSV_DIR = r"C:\Users\me\Downloads\cv-corpus-26.0-2026-06-12\hi"
MANUAL_TSV_DIR = None

# --- Project folders ---------------------------------------------------------
PROJECT_ROOT   = Path("project")
DATASET_DIR    = PROJECT_ROOT / "dataset"
FEATURES_DIR   = PROJECT_ROOT / "features"
MEL_IMAGES_DIR = PROJECT_ROOT / "mel_images"
SPLITS_DIR     = PROJECT_ROOT / "splits"
MODELS_DIR     = PROJECT_ROOT / "models"
RESULTS_DIR    = PROJECT_ROOT / "results"

for d in [PROJECT_ROOT, DATASET_DIR, FEATURES_DIR, MEL_IMAGES_DIR,
          MEL_IMAGES_DIR / "train", MEL_IMAGES_DIR / "val", MEL_IMAGES_DIR / "test",
          SPLITS_DIR, MODELS_DIR, RESULTS_DIR]:
    d.mkdir(parents=True, exist_ok=True)

# --- Class definition --------------------------------------------------------
AGE_CLASSES = ["teens", "twenties", "thirties", "fourties"]

# --- Audio preprocessing -----------------------------------------------------
SR = 16000                    # reference sample rate used EVERYWHERE (Section 2.2 item 6 / 2.3)
MAX_DURATION_SEC = None

# --- 180-dim feature config (Section 2.2 item 3) -----------------------------
N_MFCC    = 40
N_MELS    = 128
N_CHROMA = 12
N_FFT_FEAT = 2048
HOP_LENGTH_FEAT = 512
FEATURE_180_VERSION = "v2-meanpool-fullclip"   # bump this if the aggregation method ever changes
```

```python
# --- Mel-spectrogram image config (for CNN) ----------------------------------
N_MELS_IMG     = 128
N_FFT_IMG      = 1024
HOP_LENGTH_IMG = 512
IMAGE_SIZE     = 224
MEL_IMAGE_VERSION = "v2-nomargin-nocbar-fullclip"   # bump this if rendering ever changes -> avoids stale-cache reus

# --- OpenSMILE config (Section 2.2 items 1-2; change here if your tutorial differs) ---
OPENSMILE_62_FEATURE_SET  = "GeMAPSv01b"   # -> opensmile.FeatureSet.GeMAPSv01b
OPENSMILE_88_FEATURE_SET  = "eGeMAPSv01b"  # -> opensmile.FeatureSet.eGeMAPSv01b  (eGeMAPSv02 also valid, see Sec. 1
OPENSMILE_FEATURE_LEVEL   = "Functionals"

# --- Speaker-independent split (Section 10) ----------------------------------
N_SPLIT_FOLDS = 7          # 5 folds -> train (~71%), 1 -> val (~14%), 1 -> test (~14%)
SPLIT_SEED = 42
SPLIT_VERSION = "v2-stratifiedgroupkfold7"   # bump if the split *method* ever changes

# --- Hardware / training config ----------------------------------------------
BATCH_SIZE   = 32
NUM_WORKERS  = 0            # 0 by default: safest in Jupyter/Windows (avoids multiprocessing issues)
CNN_EPOCHS   = 30
CNN_PATIENCE = 6            # early-stopping patience, monitored on validation Macro F1
LR = 1e-3
USE_AMP = True              # only actually engages if CUDA is available

# --- Reproducibility ---------------------------------------------------------
GLOBAL_SEED = 42

# --- Dev-mode safety valve ---------------------------------------------------
# Smoke-test the whole pipeline on a small subset before a full run. None = full dataset.
MAX_CLIPS_PER_CLASS = None

print("Configuration loaded. Project root:", PROJECT_ROOT.resolve())
```

```python
# ============================================================
#  DATASET LOADING AND FILTERING
# ============================================================
AGE_CLASSES = ["teens", "twenties", "thirties", "fourties"]
def load_tsv(path):
    return pd.read_csv(path, sep="\t", low_memory=False)

validated_df = load_tsv(tsv_dir / "validated.tsv")
print("validated.tsv (base pool):", validated_df.shape)
print("Columns:", list(validated_df.columns))

n_before = len(validated_df)

df = validated_df.copy()
df["age"] = df["age"].astype(str).str.strip().str.lower()

# 1) keep only the 4 required classes
df = df[df["age"].isin(AGE_CLASSES)].copy()
n_after_age_filter = len(df)
# 2) drop rows missing essential fields
df = df.dropna(subset=["path", "client_id", "age"]).copy()
# 3) drop rows whose audio file does not actually exist on disk
df["abs_path"] = df["path"].apply(lambda p: str(clips_dir / p))
exists_mask = df["abs_path"].apply(os.path.exists)
n_missing_files = int((~exists_mask).sum())
df = df[exists_mask].copy()

# 4) drop exact duplicate clip entries
df = df.drop_duplicates(subset=["path"]).copy()

# 5) exclude speakers with inconsistent age labels (Section 2.1 correction)
age_per_speaker = df.groupby("client_id")["age"].nunique()
inconsistent_speakers = age_per_speaker[age_per_speaker > 1].index.tolist()
if inconsistent_speakers:
    inconsistent_rows = df[df["client_id"].isin(inconsistent_speakers)]
    inconsistent_rows.to_csv(RESULTS_DIR / "excluded_inconsistent_age_speakers.csv", index=False)
    print(f"⚠ {len(inconsistent_speakers)} speaker(s) have inconsistent age labels across clips -- "
          f"excluded ({len(inconsistent_rows)} clips). Logged to "
          f"{RESULTS_DIR / 'excluded_inconsistent_age_speakers.csv'}")
    df = df[~df["client_id"].isin(inconsistent_speakers)].copy()
else:
    print("No speakers with inconsistent age labels found.")
```

Dataset Loading & Filtering: Loads the dataset and retains valid age classes, audio files, and consistent speaker labels.

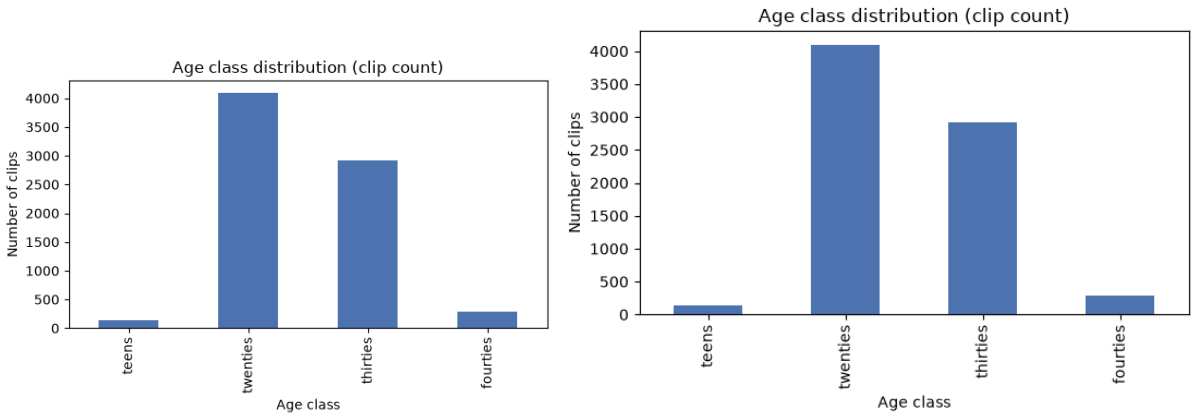
The Hindi Common Voice dataset was filtered to four age categories: teens, twenties, thirties and forties.

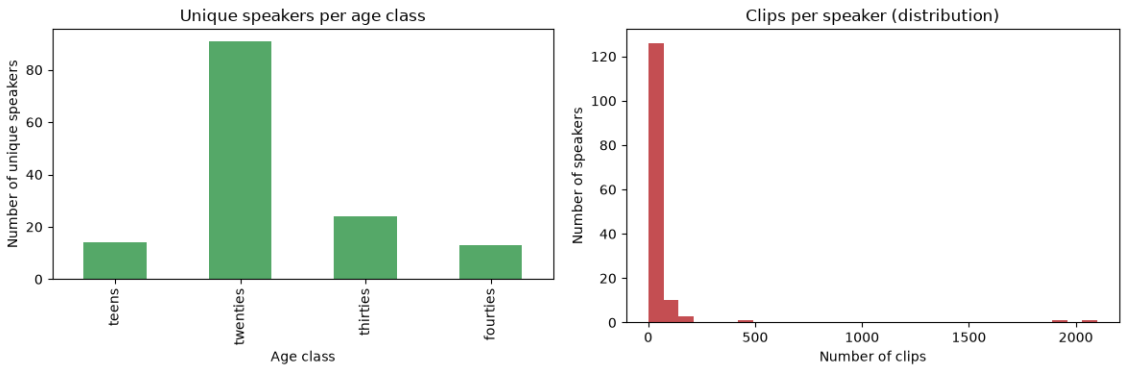
```
validated.tsv (base pool): (11131, 13)
Columns: ['client_id', 'path', 'sentence_id', 'sentence', 'sentence_domain', 'up_votes', 'down_votes', 'age', 'gender',
⚠ 1 speaker(s) have inconsistent age labels across clips -- excluded (3 clips). Logged to project\results\excluded_inco

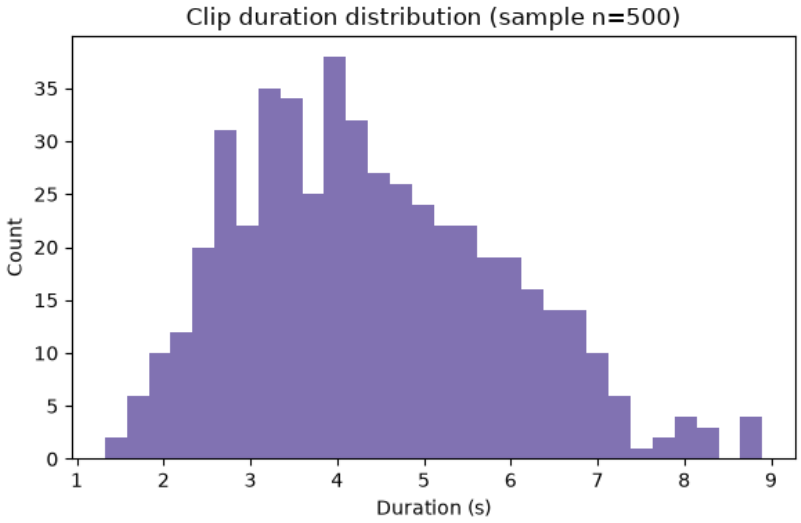
Records before filtering (validated.tsv): 11131
Records after age-class filtering to ['teens', 'twenties', 'thirties', 'fourties']: 7429
Rows dropped for missing audio file on disk: 0
Records after all filtering (existing audio, deduped, consistent-age speakers): 7426

Actual class counts (computed from data, not assumed from any datasheet):
age
teens        132
twenties    4098
thirties    2914
fourties     282
Name: count, dtype: int64

Saved filtered pool -> project\features\filtered_pool.csv
```







The dataset contains sufficient variation in age classes and recording durations for age classification, but its **uneven class and speaker distribution makes speaker-independent splitting and imbalance handling essential** for obtaining reliable results.

```python
# ============================================================
# SPEAKER-INDEPENDENT STRATIFIED GROUPED SPLIT
# ============================================================
age_encoder_full = LabelEncoder().fit(AGE_CLASSES)
y_all = age_encoder_full.transform(df["age"].values)
groups_all = df["client_id"].values

sgkf = StratifiedGroupKFold(n_splits=N_SPLIT_FOLDS, shuffle=True, random_state=SPLIT_SEED)

# Each sample gets a fold_id = index of the fold in which it appears in the HELD-OUT portion.
# Because StratifiedGroupKFold guarantees non-overlapping groups across folds' held-out sets,
# this defines a fixed, one-to-one speaker -> fold_id mapping.
fold_id = np.full(len(df), -1, dtype=int)
for i, (_, holdout_idx) in enumerate(sgkf.split(df, y_all, groups_all)):
    fold_id[holdout_idx] = i
assert (fold_id >= 0).all(), "Every sample must be assigned to exactly one fold."
df["_fold_id"] = fold_id

# Sanity: a speaker must map to exactly one fold (guaranteed by StratifiedGroupKFold, verified anyway)
speaker_fold_counts = df.groupby("client_id")["_fold_id"].nunique()
assert (speaker_fold_counts == 1).all(), "A speaker was assigned to more than one fold -- this should be impossible."

overall_dist = df["age"].value_counts(normalize=True).reindex(AGE_CLASSES).values

fold_stats = []
for i in range(N_SPLIT_FOLDS):
    fold_df = df[df["_fold_id"] == i]
    counts = fold_df["age"].value_counts().reindex(AGE_CLASSES).fillna(0)
    props = (counts / counts.sum()).values
    all_classes_present = (counts > 0).all()
    l1_dist = np.abs(props - overall_dist).sum()
    fold_stats.append({"fold": i, "n_clips": len(fold_df), "n_speakers": fold_df["client_id"].nunique(),
                       "all_classes_present": bool(all_classes_present), "l1_dist_to_overall": l1_dist})

fold_stats_df = pd.DataFrame(fold_stats).sort_values("l1_dist_to_overall")
print("Per-fold stats (sorted by closeness to overall class distribution):")
print(fold_stats_df.to_string(index=False))

eligible = fold_stats_df[fold_stats_df["all_classes_present"]]
assert len(eligible) >= 2, "Fewer than 2 folds contain all 4 classes -- cannot pick val/test folds."
```

Speaker-Independent Split: Groups samples by speaker and assigns each speaker to only one fold while preserving age-class distribution.

```python
fold_stats_df = pd.DataFrame(fold_stats).sort_values("l1_dist_to_overall")
print("Per-fold stats (sorted by closeness to overall class distribution):")
print(fold_stats_df.to_string(index=False))

eligible = fold_stats_df[fold_stats_df["all_classes_present"]]
assert len(eligible) >= 2, "Fewer than 2 folds contain all 4 classes -- cannot pick val/test folds."

test_fold = int(eligible.iloc[0]["fold"])
val_fold = int(eligible[eligible["fold"] != test_fold].iloc[0]["fold"])
train_folds = [i for i in range(N_SPLIT_FOLDS) if i not in (test_fold, val_fold)]

print(f"\nDeterministically chosen -> test fold: {test_fold} | val fold: {val_fold} | train folds: {train_folds}")

train_split = df[df["_fold_id"].isin(train_folds)].drop(columns=["_fold_id"]).reset_index(drop=True)
val_split   = df[df["_fold_id"] == val_fold].drop(columns=["_fold_id"]).reset_index(drop=True)
test_split  = df[df["_fold_id"] == test_fold].drop(columns=["_fold_id"]).reset_index(drop=True)

train_speakers = set(train_split["client_id"])
val_speakers   = set(val_split["client_id"])
test_speakers  = set(test_split["client_id"])

print("\nUnique speakers -> train:", len(train_speakers), " val:", len(val_speakers), " test:", len(test_speakers))
print("Clips           -> train:", len(train_split),   " val:", len(val_split),   " test:", len(test_split))

print("\nAge distribution (train):\n", train_split["age"].value_counts().reindex(AGE_CLASSES))
print("\nAge distribution (val):\n",   val_split["age"].value_counts().reindex(AGE_CLASSES))
print("\nAge distribution (test):\n",  test_split["age"].value_counts().reindex(AGE_CLASSES))

for split_name, split_df in [("train", train_split), ("val", val_split), ("test", test_split)]:
    missing = set(AGE_CLASSES) - set(split_df["age"].unique())
    assert not missing, f"{split_name} split is missing age class(es): {sorted(missing)}"

train_split.to_csv(SPLITS_DIR / "train_split.csv", index=False)
val_split.to_csv(SPLITS_DIR / "val_split.csv", index=False)
test_split.to_csv(SPLITS_DIR / "test_split.csv", index=False)
print(f"\nSaved split files (method={SPLIT_VERSION}) to", SPLITS_DIR.resolve())
```

- **Train/Validation/Test Split:** Selects separate speaker groups for training, validation, and testing while maintaining all four age classes.
- **Final Verification & Export:** Verifies class coverage and speaker independence, then saves the three splits as CSV files.

```
Per-fold stats (sorted by closeness to overall class distribution):
 fold  n_clips  n_speakers  all_classes_present  l1_dist_to_overall
    6      601          48                 True            0.297898
    4      524          18                 True            0.345879
    2      747          14                 True            0.353075
    3      525          17                 True            0.373382
    5      603          21                 True            0.419034
    0     2217          11                 True            0.789860
    1     2209          13                 True            0.942668

Deterministically chosen -> test fold: 6 | val fold: 4 | train folds: [0, 1, 2, 3, 5]

Unique speakers -> train: 76  val: 18  test: 48
Clips           -> train: 6301  val: 524  test: 601

Age distribution (train):
 age
teens         78
twenties    3348
thirties    2652
fourties     223
Name: count, dtype: int64

Age distribution (val):
 age
teens        44
twenties    341
thirties    115
fourties     24
Name: count, dtype: int64
```

11. Leakage verification

Hard assertions, exactly as specified in the assignment. The notebook stops here if any of these ever fail.

```python
# ============================================================
# 11. LEAKAGE VERIFICATION
# ============================================================
print("train ∩ val  :", train_speakers & val_speakers)
print("train ∩ test :", train_speakers & test_speakers)
print("val ∩ test   :", val_speakers & test_speakers)

assert train_speakers.isdisjoint(val_speakers),  "LEAKAGE: some speakers appear in both train and val!"
assert train_speakers.isdisjoint(test_speakers), "LEAKAGE: some speakers appear in both train and test!"
assert val_speakers.isdisjoint(test_speakers),   "LEAKAGE: some speakers appear in both val and test!"

# Every class must be present in every split (checked again here, defensively)
for name, split in [("train", train_split), ("val", val_split), ("test", test_split)]:
    present = set(split["age"].unique())
    missing = set(AGE_CLASSES) - present
    assert not missing, f"{name} split is missing classes: {missing}"

print("\n  Speaker-independence verified: no client_id appears in more than one split.")
print("  All 4 classes present in train, validation, and test splits.")
```

Python

```
train ∩ val  : set()
train ∩ test : set()
val ∩ test   : set()

✅ Speaker-independence verified: no client_id appears in more than one split.
✅ All 4 classes present in train, validation, and test splits.
```

OpenSMILE 62 and 88 features

```python
# ============================================================
# 13. OPENSMILE 62-FEATURE EXTRACTION
# ============================================================
if HAVE_OPENSMILE:
    smile_62 = opensmile.Smile(
        feature_set=getattr(opensmile.FeatureSet, OPENSMILE_62_FEATURE_SET),
        feature_level=getattr(opensmile.FeatureLevel, OPENSMILE_FEATURE_LEVEL),
    )
    print("opensmile version:", opensmile.__version__)
    print("62-feature config -> FeatureSet:", OPENSMILE_62_FEATURE_SET, "| FeatureLevel:", OPENSMILE_FEATURE_LEVEL)
    print("Measured output dimensionality:", len(smile_62.feature_names))
    assert len(smile_62.feature_names) == 62, \
        f"Configured '62' feature set actually produced {len(smile_62.feature_names)} features -- " \
        f"check OPENSMILE_62_FEATURE_SET in Section 5 against your course tutorial."
    print(" Verified: 62-dim output.")

    def extract_opensmile(dataframe, smile_obj, out_path, expected_dim, label):
        if out_path.exists():
            print(f"[cache hit] Loading cached {label} features from {out_path}")
            return pd.read_csv(out_path)
        rows, fail_count = [], 0
        for _, r in tqdm(dataframe.iterrows(), total=len(dataframe), desc=f"Extracting {label}"):
            y = load_audio_safe(r["abs_path"])
            if y is None:
                fail_count += 1
                continue
            try:
                feats = smile_obj.process_signal(y, SR)
                vec = feats.iloc[0].values.astype(np.float32)
                if len(vec) != expected_dim:
                    raise ValueError(f"Expected {expected_dim} dims, got {len(vec)}")
                row = {"clip_id": r["clip_id"], "client_id": r["client_id"], "age": r["age"]}
                row.update({f"f{i}": v for i, v in enumerate(vec)})
                rows.append(row)
            except Exception as e:
                fail_count += 1
                failed_audio_records.append({"path": r["abs_path"], "error": f"{label}: {type(e).__name__}: {e}"})
        result_df = pd.DataFrame(rows)
        result_df.to_csv(out_path, index=False)
        print(f"{label}: succeeded={len(result_df)}  failed={fail_count}  -> saved to {out_path}")
        return result_df
```

```
opensmile version: 2.6.0
62-feature config -> FeatureSet: GeMAPSv01b | FeatureLevel: Functionals
Measured output dimensionality: 62
✅ Verified: 62-dim output.
Extracting OpenSMILE-62: 100%|██████████| 7426/7426 [11:46<00:00, 10.51it/s]
OpenSMILE-62: succeeded=7426  failed=0  -> saved to project\features\opensmile_62_GeMAPSv01b_Functionals.csv
opensmile_62 shape: (7426, 65)
```

```python
# ============================================================
# 14. OPENSMILE 88-FEATURE EXTRACTION
# ============================================================
if HAVE_OPENSMILE:
    smile_88 = opensmile.Smile(
        feature_set=getattr(opensmile.FeatureSet, OPENSMILE_88_FEATURE_SET),
        feature_level=getattr(opensmile.FeatureLevel, OPENSMILE_FEATURE_LEVEL),
    )
    print("88-feature config -> FeatureSet:", OPENSMILE_88_FEATURE_SET, "| FeatureLevel:", OPENSMILE_FEATURE_LEVEL)
    print("Measured output dimensionality:", len(smile_88.feature_names))
    assert len(smile_88.feature_names) == 88, \
        f"Configured '88' feature set actually produced {len(smile_88.feature_names)} features -- " \
        f"check OPENSMILE_88_FEATURE_SET in Section 5 against your course tutorial."
    print("Verified: 88-dim output.")

    opensmile_88_path = FEATURES_DIR / f"opensmile_88_{OPENSMILE_88_FEATURE_SET}_{OPENSMILE_FEATURE_LEVEL}.csv"
    opensmile_88_df = extract_opensmile(df, smile_88, opensmile_88_path, 88, "OpenSMILE-88")
    assert opensmile_88_df.shape[1] - 3 == 88
    print("opensmile_88 shape:", opensmile_88_df.shape)
else:
    opensmile_88_df = None
    print("Skipped: opensmile not installed.")
```

```
88-feature config -> FeatureSet: eGeMAPSv01b | FeatureLevel: Functionals
Measured output dimensionality: 88
Verified: 88-dim output.
Extracting OpenSMILE-88: 100%|██████████| 7426/7426 [12:40<00:00,  9.77it/s]
OpenSMILE-88: succeeded=7426  failed=0  -> saved to project\features\opensmile_88_eGeMAPSv01b_Functionals.csv
opensmile_88 shape: (7426, 91)
```

- **GeMAPSv01b produced 62 features, while eGeMAPSv01b produced 88 features**

180-Dimensional Audio Feature Extraction

```python
def extract_180_features(y):
    if len(y) < N_FFT_FEAT:
        y = np.pad(y, (0, N_FFT_FEAT - len(y)))

    mfcc = librosa.feature.mfcc(y=y, sr=SR, n_mfcc=N_MFCC, n_fft=N_FFT_FEAT, hop_length=HOP_LENGTH_FEAT)
    mel  = librosa.feature.melspectrogram(y=y, sr=SR, n_mels=N_MELS, n_fft=N_FFT_FEAT, hop_length=HOP_LENGTH_FEAT)
    mel_db = librosa.power_to_db(mel, ref=np.max)
    chroma = librosa.feature.chroma_stft(y=y, sr=SR, n_chroma=N_CHROMA, n_fft=N_FFT_FEAT, hop_length=HOP_LENGTH_FEAT)

    mfcc_vec   = mfcc.mean(axis=1)      # (40,)  -- mean over time
    mel_vec    = mel_db.mean(axis=1)    # (128,) -- mean over time
    chroma_vec = chroma.mean(axis=1)    # (12,)  -- mean over time

    vec = np.concatenate([mfcc_vec, mel_vec, chroma_vec]).astype(np.float32)
    assert vec.shape[0] == N_MFCC + N_MELS + N_CHROMA == 180
    return vec

out_path_180 = FEATURES_DIR / f"features_180_{FEATURE_180_VERSION}.csv"
if out_path_180.exists():
    print(f"[cache hit] Loading cached 180-dim features from {out_path_180}")
    features_180_df = pd.read_csv(out_path_180)
else:
    rows, fail_count = [], 0
    for _, r in tqdm(df.iterrows(), total=len(df), desc="Extracting 180-dim features"):
        y = load_audio_safe(r["abs_path"])
        if y is None:
            fail_count += 1
            continue
        vec = extract_180_features(y)
        row = {"clip_id": r["clip_id"], "client_id": r["client_id"], "age": r["age"]}
        row.update({f"f{i}": v for i, v in enumerate(vec)})
        rows.append(row)
    features_180_df = pd.DataFrame(rows)
    features_180_df.to_csv(out_path_180, index=False)
    print(f"180-dim: succeeded={len(features_180_df)}  failed={fail_count}  -> saved to {out_path_180}")

flush_failed_audio_log()
X_check = features_180_df.drop(columns=["clip_id", "client_id", "age"])
assert X_check.shape[1] == 180, f"Expected 180 columns, got {X_check.shape[1]}"
print("\n  features_180 shape (excluding id/age columns):", X_check.shape)
```

```
Extracting 180-dim features: 100%|██████████| 7426/7426 [04:07<00:00, 29.97it/s]
180-dim: succeeded=7426  failed=0  -> saved to project\features\features_180_v2-meanpool-fullclip.csv
Failed (cumulative, unique paths): 0

  features_180 shape (excluding id/age columns): (7426, 180)
```

16. Leakage-safe ML preprocessing

Each feature table is joined to the speaker-independent split (Section 10) by `clip_id`. Two scaled views are produced per feature set:

- a **tuning view**: `StandardScaler` fit on **train only**, applied to train/val — used exclusively for hyperparameter selection (Section 17);
- a **final view**: `StandardScaler` refit on **train+validation combined**, applied to train+val/test — used only for the final model fit and the one-shot test evaluation (Section 18).

This matches the assignment's explicit requirement: *"after selecting hyperparameters, refit the final classical ML model on train+validation... refit the scaler on train+validation only... then evaluate the held-out test speakers once per model."*

```
# ============================================================
# 16. LEAKAGE-SAFE ML PREPROCESSING
# ============================================================
label_encoder = age_encoder_full  # already fit on AGE_CLASSES (Section 10)

def prepare_feature_set(feat_df, name):
    feat_cols = [c for c in feat_df.columns if c.startswith("f")]

    def subset(split_df):
        merged = split_df[["clip_id"]].merge(feat_df, on="clip_id", how="inner")
        X = merged[feat_cols].values.astype(np.float32)
        y = label_encoder.transform(merged["age"].values)
        groups = merged["client_id"].values
        return X, y, groups

    X_train, y_train, g_train = subset(train_split)
    X_val,   y_val,   g_val   = subset(val_split)
    X_test,  y_test,  g_test  = subset(test_split)

    # leakage guard, repeated at the feature-table level
    assert set(g_train).isdisjoint(set(g_val))
    assert set(g_train).isdisjoint(set(g_test))
    assert set(g_val).isdisjoint(set(g_test))

    # --- tuning view: scaler fit on TRAIN ONLY ---
    tuning_scaler = StandardScaler().fit(X_train)
```

- **Leakage-Safe Preprocessing:** Joins features with speaker-independent splits and scales them using training data only, with a final scaler refitted on train + validation before one-time test evaluation.

```
feature_sets = {}
if opensmile_62_df is not None:
    feature_sets["OpenSMILE 62"] = prepare_feature_set(opensmile_62_df, "OpenSMILE 62")
if opensmile_88_df is not None:
    feature_sets["OpenSMILE 88"] = prepare_feature_set(opensmile_88_df, "OpenSMILE 88")
feature_sets["180 combined"] = prepare_feature_set(features_180_df, "180 combined")

print("\nPrepared feature sets:", list(feature_sets.keys()))
```

```
[OpenSMILE 62] train=(6301, 62) val=(524, 62) test=(601, 62) (train+val combined for final fit: (6825, 62))
[OpenSMILE 88] train=(6301, 88) val=(524, 88) test=(601, 88) (train+val combined for final fit: (6825, 88))
[180 combined] train=(6301, 180) val=(524, 180) test=(601, 180) (train+val combined for final fit: (6825, 180))

Prepared feature sets: ['OpenSMILE 62', 'OpenSMILE 88', '180 combined']
```

```python
# ============================================================
# 17. ML HYPERPARAMETER TUNING  (train -> val only, test untouched)
# ============================================================
SVM_GRID = [{"C": c, "gamma": g, "kernel": "rbf"} for c in [1, 10] for g in ["scale", 0.01]]
RF_GRID  = [{"n_estimators": n, "max_depth": d, "min_samples_split": s}
            for n in [200, 400] for d in [None, 20] for s in [2]]
XGB_GRID = [{"n_estimators": n, "max_depth": d, "learning_rate": lr}
            for n in [200, 400] for d in [4, 6] for lr in [0.1]]

MODEL_NAMES = ["SVM", "Random Forest", "XGBoost"]

def macro_f1(y_true, y_pred):
    return f1_score(y_true, y_pred, average="macro")

def class_weights_dict(y_train):
    classes = np.unique(y_train)
    weights = compute_class_weight("balanced", classes=classes, y=y_train)
    return dict(zip(classes, weights))

def fit_one(model_name, params, X, y, sample_weight=None):
    if model_name == "SVM":
        clf = SVC(class_weight="balanced", random_state=GLOBAL_SEED, **params)
        clf.fit(X, y)
    elif model_name == "Random Forest":
        clf = RandomForestClassifier(class_weight="balanced", random_state=GLOBAL_SEED, n_jobs=-1, **params)
        clf.fit(X, y)
    else:  # XGBoost or fallback
        if HAVE_XGBOOST:
            clf = xgb.XGBClassifier(random_state=GLOBAL_SEED, eval_metric="mlogloss", n_jobs=-1, **params)
            clf.fit(X, y, sample_weight=sample_weight)
        else:
            clf = HistGradientBoostingClassifier(random_state=GLOBAL_SEED, max_iter=params["n_estimators"],
                                                 max_depth=params["max_depth"], learning_rate=params["learning_rate"])
            clf.fit(X, y, sample_weight=sample_weight)
    return clf

def tune_hyperparams(model_name, data):
    X_train, y_train = data["X_train_tune"], data["y_train"]
    X_val,   y_val   = data["X_val_tune"],   data["y_val"]
    cw = class_weights_dict(y_train)
    sw_train = np.array([cw[c] for c in y_train])
```

- **ML Hyperparameter Tuning:** Tunes SVM, Random Forest, and XGBoost using training data and validates configurations on the validation set while keeping the test set untouched.

```
=== Tuning on feature set: OpenSMILE 62 (train -> val) ===
  best SVM: val macro-F1=0.2619  params={'C': 10, 'gamma': 'scale', 'kernel': 'rbf'}
  best Random Forest: val macro-F1=0.2204  params={'n_estimators': 400, 'max_depth': 20, 'min_samples_split': 2}
  best XGBoost: val macro-F1=0.2499  params={'n_estimators': 200, 'max_depth': 4, 'learning_rate': 0.1}

=== Tuning on feature set: OpenSMILE 88 (train -> val) ===
  best SVM: val macro-F1=0.1942  params={'C': 10, 'gamma': 'scale', 'kernel': 'rbf'}
  best Random Forest: val macro-F1=0.2176  params={'n_estimators': 400, 'max_depth': None, 'min_samples_split': 2}
  best XGBoost: val macro-F1=0.2391  params={'n_estimators': 400, 'max_depth': 4, 'learning_rate': 0.1}

=== Tuning on feature set: 180 combined (train -> val) ===
  best SVM: val macro-F1=0.2027  params={'C': 1, 'gamma': 0.01, 'kernel': 'rbf'}
  best Random Forest: val macro-F1=0.1827  params={'n_estimators': 400, 'max_depth': 20, 'min_samples_split': 2}
  best XGBoost: val macro-F1=0.2033  params={'n_estimators': 200, 'max_depth': 4, 'learning_rate': 0.1}
```

```python
# ============================================================
# 18. FINAL ML REFIT (train+val) + ONE-SHOT TEST EVALUATION
# ============================================================
ml_results = []

for (fs_name, model_name), (params, val_score) in best_hyperparams.items():
    data = feature_sets[fs_name]
    X_trainval, y_trainval = data["X_trainval_final"], data["y_trainval"]
    X_test, y_test = data["X_test_final"], data["y_test"]

    cw = class_weights_dict(y_trainval)
    sw_trainval = np.array([cw[c] for c in y_trainval])

    final_model = fit_one(model_name, params, X_trainval, y_trainval, sample_weight=sw_trainval)
    y_pred = final_model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    prec_macro = precision_score(y_test, y_pred, average="macro", zero_division=0)
    rec_macro  = recall_score(y_test, y_pred, average="macro", zero_division=0)
    f1_macro   = f1_score(y_test, y_pred, average="macro", zero_division=0)
    f1_weighted = f1_score(y_test, y_pred, average="weighted", zero_division=0)

    print(f"\n--- {fs_name} + {model_name} (final, train+val refit, test eval) ---")
    print(classification_report(y_test, y_pred, target_names=label_encoder.classes_, zero_division=0))
    cm = confusion_matrix(y_test, y_pred, labels=range(len(AGE_CLASSES)))
    fig, ax = plt.subplots(figsize=(4, 4))
    ConfusionMatrixDisplay(cm, display_labels=label_encoder.classes_).plot(ax=ax, colorbar=False)
    ax.set_title(f"{fs_name} + {model_name}")
    plt.tight_layout(); plt.show()

    ml_results.append({
        "Feature Set": fs_name, "Model": model_name,
        "Accuracy": acc, "Macro Precision": prec_macro, "Macro Recall": rec_macro,
        "Macro F1": f1_macro, "Weighted F1": f1_weighted,
        "Best Params (tuned on val)": json.dumps(params), "Val Macro F1 (tuning stage)": val_score,
    })

ml_results_df = pd.DataFrame(ml_results)
ml_results_df.to_csv(RESULTS_DIR / "ml_results.csv", index=False)
print("\n=== ML results (final, one-shot test evaluation) ===")
ml_results_df[["Feature Set", "Model", "Accuracy", "Macro Precision", "Macro Recall", "Macro F1", "Weighted F1"]]
```

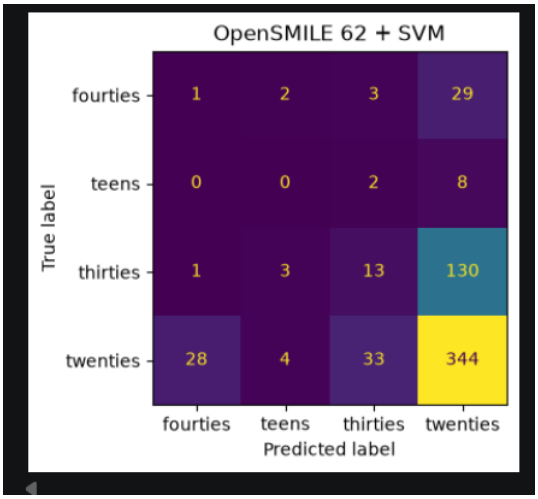
- **Final ML Evaluation:** Refits the best models on train + validation data, evaluates once on the unseen test set, and records accuracy, precision, recall, and F1-score with confusion matrices.

Outputs:

```
--- OpenSMILE 62 + SVM (final, train+val refit, test eval) ---
              precision    recall  f1-score   support

    fourties       0.03      0.03      0.03        35
       teens       0.00      0.00      0.00        10
    thirties       0.25      0.09      0.13       147
    twenties       0.67      0.84      0.75       409

    accuracy                           0.60       601
   macro avg       0.24      0.24      0.23       601
weighted avg       0.52      0.60      0.54       601
```

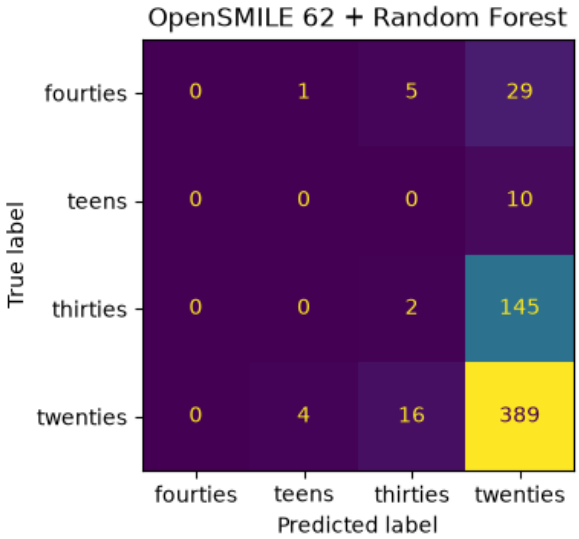


- **SVM Results:** The model achieved **60% accuracy and 0.23 macro F1**, with strongest performance for the **twenties** class (F1 = 0.75), while teens and forties were poorly classified.

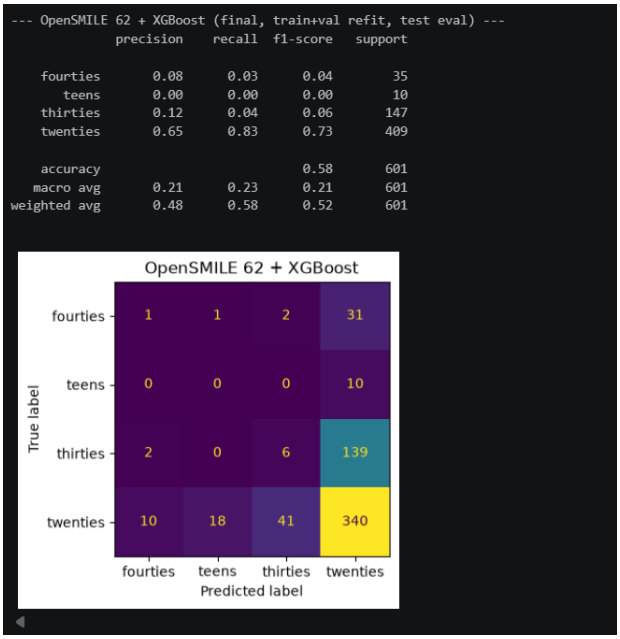
```
--- OpenSMILE 62 + Random Forest (final, train+val refit, test eval) ---
              precision    recall  f1-score   support

    fourties       0.00      0.00      0.00        35
       teens       0.00      0.00      0.00        10
    thirties       0.09      0.01      0.02       147
    twenties       0.68      0.95      0.79       409

    accuracy                           0.65       601
   macro avg       0.19      0.24      0.20       601
weighted avg       0.48      0.65      0.54       601
```



- **Random Forest Results:** The model achieved **65% accuracy and 0.20 macro F1**, performing strongly on the **twenties** class (F1 = 0.79) but poorly on teens, thirties, and forties.

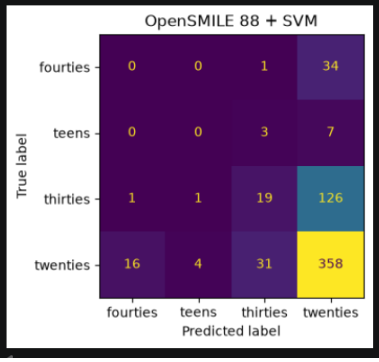


- **XGBoost Results:** The model achieved **58% accuracy and 0.21 macro F1**, with its best performance on the **twenties** class (F1 = 0.73), while teens, thirties, and forties showed low classification performance.

```
--- OpenSMILE 88 + SVM (final, train+val refit, test eval) ---
              precision    recall  f1-score   support

    fourties       0.00      0.00      0.00        35
       teens       0.00      0.00      0.00        10
    thirties       0.35      0.13      0.19       147
    twenties       0.68      0.88      0.77       409

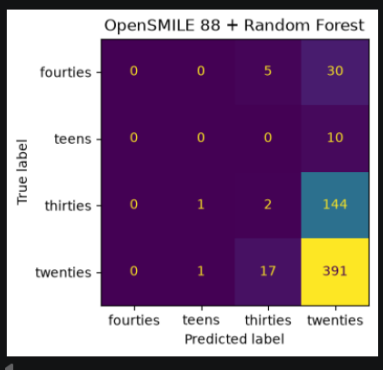
    accuracy                           0.63       601
   macro avg       0.26      0.25      0.24       601
weighted avg       0.55      0.63      0.57       601
```




```
--- OpenSMILE 88 + Random Forest (final, train+val refit, test eval) ---
              precision    recall  f1-score   support

    fourties       0.00      0.00      0.00        35
       teens       0.00      0.00      0.00        10
    thirties       0.08      0.01      0.02       147
    twenties       0.68      0.96      0.79       409

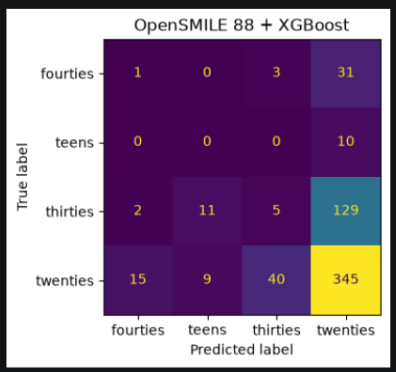
    accuracy                           0.65       601
   macro avg       0.19      0.24      0.20       601
weighted avg       0.48      0.65      0.55       601
```




```
--- OpenSMILE 88 + XGBoost (final, train+val refit, test eval) ---
              precision    recall  f1-score   support

    fourties       0.06      0.03      0.04        35
       teens       0.00      0.00      0.00        10
    thirties       0.10      0.03      0.05       147
    twenties       0.67      0.84      0.75       409

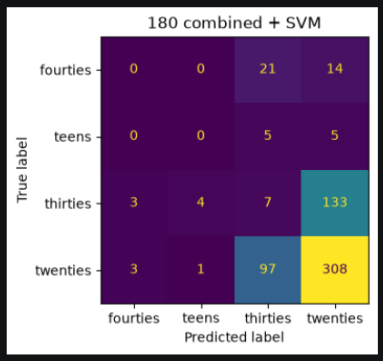
    accuracy                           0.58       601
   macro avg       0.21      0.23      0.21       601
weighted avg       0.48      0.58      0.52       601
```




```
--- 180 combined + SVM (final, train+val refit, test eval) ---
              precision    recall  f1-score   support

    fourties       0.00      0.00      0.00        35
       teens       0.00      0.00      0.00        10
    thirties       0.05      0.05      0.05       147
    twenties       0.67      0.75      0.71       409

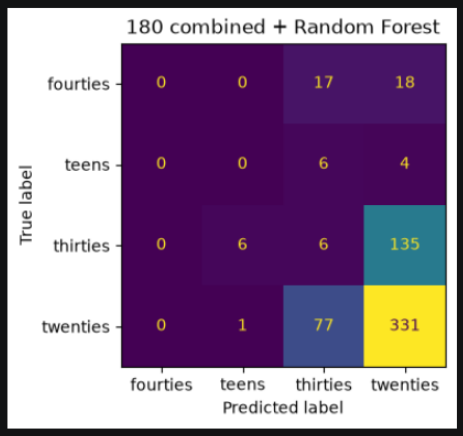
    accuracy                           0.52       601
   macro avg       0.18      0.20      0.19       601
weighted avg       0.47      0.52      0.49       601
```



```
--- 180 combined + Random Forest (final, train+val refit, test
              precision    recall  f1-score   support

    fourties       0.00      0.00      0.00        35
       teens       0.00      0.00      0.00        10
    thirties       0.06      0.04      0.05       147
    twenties       0.68      0.81      0.74       409

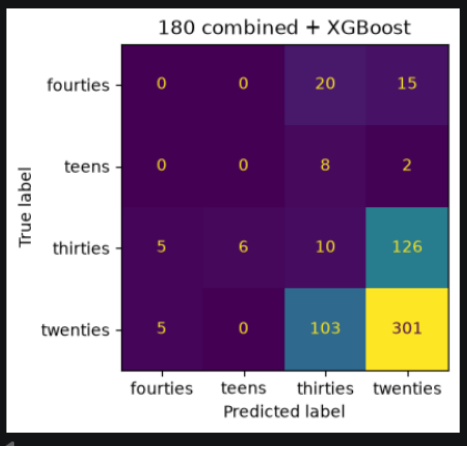
    accuracy                           0.56       601
   macro avg       0.18      0.21      0.20       601
weighted avg       0.48      0.56      0.51       601
```




```
--- 180 combined + XGBoost (final, train+val refit, test eval) ---
              precision    recall  f1-score   support

    fourties       0.00      0.00      0.00        35
       teens       0.00      0.00      0.00        10
    thirties       0.07      0.07      0.07       147
    twenties       0.68      0.74      0.71       409

    accuracy                           0.52       601
   macro avg       0.19      0.20      0.19       601
weighted avg       0.48      0.52      0.50       601
```




```
=== ML results (final, one-shot test evaluation) ===
```

	Feature Set	Model	Accuracy	Macro Precision	Macro Recall	Macro F1	Weighted F1
0	OpenSMILE 62	SVM	0.595674	0.240356	0.239521	0.227477	0.542830
1	OpenSMILE 62	Random Forest	0.650582	0.191460	0.241176	0.203948	0.544914
2	OpenSMILE 62	XGBoost	0.577371	0.213305	0.225171	0.208715	0.515531
3	OpenSMILE 88	SVM	0.627288	0.258439	0.251139	0.238913	0.567934
4	OpenSMILE 88	Random Forest	0.653910	0.190833	0.242399	0.204527	0.546551
5	OpenSMILE 88	XGBoost	0.584027	0.207406	0.226526	0.208943	0.522931
6	180 combined	SVM	0.524126	0.180853	0.200169	0.189851	0.494765
7	180 combined	Random Forest	0.560732	0.183721	0.212527	0.196362	0.513845
8	180 combined	XGBoost	0.517471	0.187212	0.200992	0.193797	0.497268

- **Overall ML Results:** Among all experiments, **OpenSMILE 88 + Random Forest** achieved the highest accuracy (**65.39%**), while **OpenSMILE 62 + Random Forest** achieved **65.06%**; however, macro F1 remained low (**≈0.20**), indicating poor performance on minority age classes.

19. Mel-spectrogram image generation (for the CNN)

Clean `IMAGE_SIZE x IMAGE_SIZE` PNGs — no axes, no axis labels, no colorbar, no title — generated from the same audio files, using the **same speaker split** as the ML experiments.

```
# ============================================================
# 19. MEL-SPECTROGRAM IMAGE GENERATION  (axes/colorbar/title-free, cached)
# ============================================================
def save_mel_image(y, out_path, size=IMAGE_SIZE):
    mel = librosa.feature.melspectrogram(y=y, sr=SR, n_mels=N_MELS_IMG, n_fft=N_FFT_IMG, hop_length=HOP_LENGTH_IMG)
    mel_db = librosa.power_to_db(mel, ref=np.max)

    dpi = 100
    fig = plt.figure(figsize=(size / dpi, size / dpi), dpi=dpi)
    ax = fig.add_axes([0, 0, 1, 1])   # fill the whole figure, zero margins
    ax.axis("off")                    # no x-axis, no y-axis, no labels
    librosa.display.specshow(mel_db, sr=SR, hop_length=HOP_LENGTH_IMG, ax=ax, cmap="magma")
    # no colorbar call, no title call -> spectrogram image only
    fig.savefig(out_path, dpi=dpi)
    plt.close(fig)

def build_mel_image_set(split_df, split_name):
    out_dir = MEL_IMAGES_DIR / split_name
    manifest_path = out_dir / f"manifest_{MEL_IMAGE_VERSION}.csv"
    if manifest_path.exists():
        print(f"[cache hit] {split_name} mel-image manifest already exists ({manifest_path})")
        return pd.read_csv(manifest_path)

    rows = []
    for _, r in tqdm(split_df.iterrows(), total=len(split_df), desc=f"Mel images ({split_name})"):
        img_path = out_dir / f"{r['clip_id']}.png"
        if not img_path.exists():
            y = load_audio_safe(r["abs_path"])
            if y is None:
                continue
```

```
        rows = []
        for _, r in tqdm(split_df.iterrows(), total=len(split_df), desc=f"Mel images ({split_name})"):
            img_path = out_dir / f"{r['clip_id']}.png"
            if not img_path.exists():
                y = load_audio_safe(r["abs_path"])
                if y is None:
                    continue
                try:
                    save_mel_image(y, img_path)
                except Exception as e:
                    failed_audio_records.append({"path": r["abs_path"], "error": f"mel_image: {type(e).__name__}: {e}"})
                    continue
            rows.append({"clip_id": r["clip_id"], "client_id": r["client_id"], "age": r["age"], "image_path": str(img_path)})

        manifest = pd.DataFrame(rows)
        manifest.to_csv(manifest_path, index=False)
        print(f"{split_name}: {len(manifest)} images -> {out_dir}")
        return manifest

    train_manifest = build_mel_image_set(train_split, "train")
    val_manifest   = build_mel_image_set(val_split, "val")
    test_manifest  = build_mel_image_set(test_split, "test")

    flush_failed_audio_log()
```

```
Mel images (train): 100%|██████████| 6301/6301 [08:04<00:00, 13.02it/s]
train: 6301 images -> project\mel_images\train
Mel images (val): 100%|██████████| 524/524 [01:19<00:00,  6.57it/s]
val: 524 images -> project\mel_images\val
Mel images (test): 100%|██████████| 601/601 [00:59<00:00, 10.05it/s]
test: 601 images -> project\mel_images\test
Failed (cumulative, unique paths): 0
```

21. 2D CNN architecture

A compact 3-block CNN (Conv-BN-ReLU-Pool × 3) + global average pooling + dropout + FC — intentionally small, since the goal is a fair representation comparison against the ML feature sets, not a large model.

```python
# ============================================================
# 21. 2D CNN ARCHITECTURE
# ============================================================
if HAVE_TORCH:
    class SimpleAgeCNN(nn.Module):
        def __init__(self, n_classes=4, dropout=0.4):
            super().__init__()
            def block(cin, cout):
                return nn.Sequential(
                    nn.Conv2d(cin, cout, kernel_size=3, padding=1),
                    nn.BatchNorm2d(cout),
                    nn.ReLU(inplace=True),
                    nn.MaxPool2d(2),
                )
            self.features = nn.Sequential(block(3, 32), block(32, 64), block(64, 128))
            self.gap = nn.AdaptiveAvgPool2d(1)
            self.dropout = nn.Dropout(dropout)
            self.fc = nn.Linear(128, n_classes)

        def forward(self, x):
            x = self.features(x)
            x = self.gap(x).flatten(1)
            x = self.dropout(x)
            return self.fc(x)

    model_cnn = SimpleAgeCNN(n_classes=len(AGE_CLASSES)).to(DEVICE)
    n_params = sum(p.numel() for p in model_cnn.parameters())
    print(model_cnn)
    print(f"\nTotal parameters: {n_params:,}")
```

```
SimpleAgeCNN(
  (features): Sequential(
    (0): Sequential(
      (0): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): ReLU(inplace=True)
      (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    )
    (1): Sequential(
      (0): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): ReLU(inplace=True)
      (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    )
    (2): Sequential(
      (0): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): ReLU(inplace=True)
      (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    )
  )
  (gap): AdaptiveAvgPool2d(output_size=1)
  (dropout): Dropout(p=0.4, inplace=False)
  (fc): Linear(in_features=128, out_features=4, bias=True)
)

Total parameters: 94,212
```

- **CNN Model Architecture:** A 3-block 2D CNN with **32, 64, and 128 filters**, Batch Normalization, ReLU, and MaxPooling, followed by global average pooling and dropout, producing **4 age-class predictions with 94,212 trainable parameters**.

22. CNN training

Class-weighted `CrossEntropyLoss`, Adam, `ReduceLROnPlateau`, early stopping and checkpointing **on validation Macro F1** (not loss alone, and never on test), mixed precision only when CUDA is available via the current `torch.amp` API.

```
# ============================================================
# 22. CNN TRAINING  (best checkpoint selected by VALIDATION MACRO F1)
# ============================================================
if HAVE_TORCH:
    train_labels_np = np.array([int(label_encoder.transform([a])[0]) for a in train_manifest["age"]])
    class_weights_arr = compute_class_weight("balanced", classes=np.arange(len(AGE_CLASSES)), y=train_labels_np)
    class_weights_t = torch.tensor(class_weights_arr, dtype=torch.float32, device=DEVICE)

    criterion = nn.CrossEntropyLoss(weight=class_weights_t)
    optimizer = torch.optim.Adam(model_cnn.parameters(), lr=LR)
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode="max", factor=0.5, patience=3)
    amp_enabled = USE_AMP and CUDA_AVAILABLE
    scaler_amp = torch.amp.GradScaler("cuda", enabled=amp_enabled)

    def run_epoch(loader, train_mode):
        model_cnn.train(train_mode)
        total_loss, all_preds, all_labels = 0.0, [], []
        for imgs, labels in loader:
            imgs, labels = imgs.to(DEVICE), labels.to(DEVICE)
            if train_mode:
                optimizer.zero_grad()
            with torch.set_grad_enabled(train_mode):
                with torch.amp.autocast("cuda", enabled=amp_enabled):
                    logits = model_cnn(imgs)
                    loss = criterion(logits, labels)
                if train_mode:
                    scaler_amp.scale(loss).backward()
                    scaler_amp.step(optimizer)
                    scaler_amp.update()
            total_loss += loss.item() * imgs.size(0)
            all_preds.extend(logits.argmax(1).detach().cpu().numpy())
            all_labels.extend(labels.cpu().numpy())
        avg_loss = total_loss / len(all_labels)
```

```
    history = {"train_loss": [], "val_loss": [], "train_acc": [], "val_acc": [], "val_macro_f1": []}
    best_val_macro_f1 = -1.0
    epochs_no_improve = 0
    best_ckpt_path = MODELS_DIR / "best_cnn_model.pth"

    for epoch in range(1, CNN_EPOCHS + 1):
        t0 = time.time()
        tr_loss, tr_acc, _ = run_epoch(train_loader, train_mode=True)
        val_loss, val_acc, val_mf1 = run_epoch(val_loader, train_mode=False)
        scheduler.step(val_mf1)

        history["train_loss"].append(tr_loss); history["val_loss"].append(val_loss)
        history["train_acc"].append(tr_acc); history["val_acc"].append(val_acc)
        history["val_macro_f1"].append(val_mf1)

        improved = val_mf1 > best_val_macro_f1
        if improved:
            best_val_macro_f1 = val_mf1
            epochs_no_improve = 0
            torch.save({"model_state": model_cnn.state_dict(), "epoch": epoch,
                        "val_loss": val_loss, "val_acc": val_acc, "val_macro_f1": val_mf1}, best_ckpt_path)
        else:
            epochs_no_improve += 1

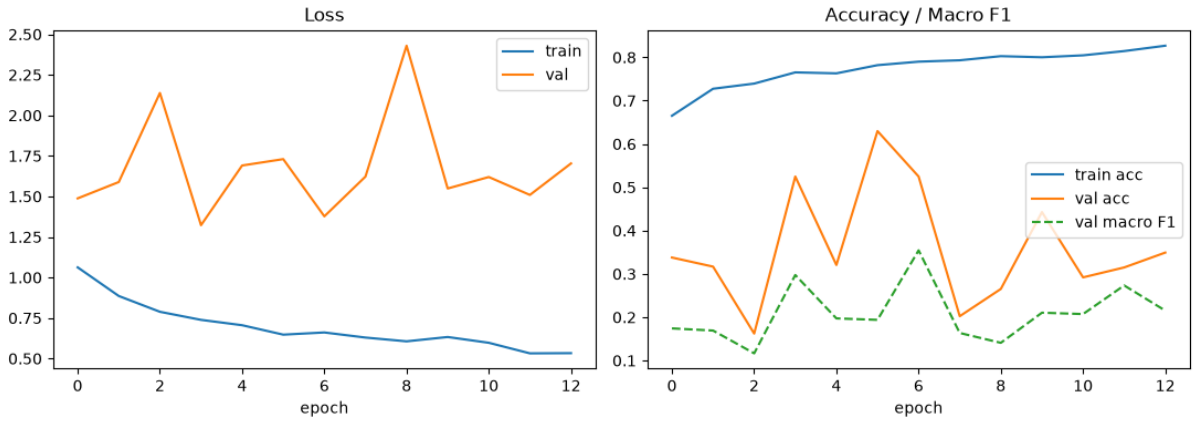
        print(f"Epoch {epoch:02d}/{CNN_EPOCHS} | train loss {tr_loss:.4f} acc {tr_acc:.4f} "
              f"| val loss {val_loss:.4f} acc {val_acc:.4f} macroF1 {val_mf1:.4f} "
              f"| {'*saved*' if improved else f'no improve ({epochs_no_improve})'} "
              f"| {time.time() - t0:.1f}s")

        if epochs_no_improve >= CNN_PATIENCE:
            print(f"Early stopping at epoch {epoch} (no val Macro F1 improvement for {CNN_PATIENCE} epochs).")
            break

    print(f"\nBest checkpoint (by validation Macro F1) saved to {best_ckpt_path}")
```

```
Epoch 01/30 | train loss 1.0624 acc 0.6651 | val loss 1.4879 acc 0.3378 macroF1 0.1742 | *saved* | 74.5s
Epoch 02/30 | train loss 0.8858 acc 0.7275 | val loss 1.5887 acc 0.3168 macroF1 0.1692 | no improve (1) | 62.6s
Epoch 03/30 | train loss 0.7884 acc 0.7394 | val loss 2.1380 acc 0.1622 macroF1 0.1166 | no improve (2) | 81.6s
Epoch 04/30 | train loss 0.7386 acc 0.7653 | val loss 1.3225 acc 0.5248 macroF1 0.2974 | *saved* | 78.6s
Epoch 05/30 | train loss 0.7056 acc 0.7631 | val loss 1.6907 acc 0.3206 macroF1 0.1972 | no improve (1) | 82.0s
Epoch 06/30 | train loss 0.6474 acc 0.7819 | val loss 1.7300 acc 0.6298 macroF1 0.1939 | no improve (2) | 66.3s
Epoch 07/30 | train loss 0.6609 acc 0.7900 | val loss 1.3771 acc 0.5248 macroF1 0.3544 | *saved* | 64.4s
Epoch 08/30 | train loss 0.6295 acc 0.7932 | val loss 1.6223 acc 0.2023 macroF1 0.1636 | no improve (1) | 65.0s
Epoch 09/30 | train loss 0.6062 acc 0.8027 | val loss 2.4295 acc 0.2653 macroF1 0.1409 | no improve (2) | 65.1s
Epoch 10/30 | train loss 0.6329 acc 0.8002 | val loss 1.5489 acc 0.4427 macroF1 0.2104 | no improve (3) | 65.0s
Epoch 11/30 | train loss 0.5972 acc 0.8046 | val loss 1.6196 acc 0.2920 macroF1 0.2073 | no improve (4) | 65.1s
Epoch 12/30 | train loss 0.5321 acc 0.8145 | val loss 1.5092 acc 0.3149 macroF1 0.2733 | no improve (5) | 64.5s
Epoch 13/30 | train loss 0.5333 acc 0.8269 | val loss 1.7035 acc 0.3492 macroF1 0.2152 | no improve (6) | 64.4s
Early stopping at epoch 13 (no val Macro F1 improvement for 6 epochs).

Best checkpoint (by validation Macro F1) saved to project\models\best_cnn_model.pth
```



- **CNN Training Results:** The CNN reached **82.9% training accuracy**, while validation accuracy peaked at **62.9%** and validation Macro F1 at **0.354**; early stopping at epoch 13 indicates **overfitting**, with the best model saved based on validation Macro F1.

23. CNN test evaluation

Loads the checkpoint selected by validation Macro F1 and evaluates it **once** on the held-out, speaker-disjoint test set.

```python
# ============================================================
# 23. CNN TEST EVALUATION  (best-by-val-macroF1 checkpoint, test set, once)
# ============================================================
if HAVE_TORCH:
    ckpt = torch.load(best_ckpt_path, map_location=DEVICE)
    model_cnn.load_state_dict(ckpt["model_state"])
    model_cnn.eval()
    print(f"Loaded best checkpoint from epoch {ckpt['epoch']} "
          f"(val_macro_f1={ckpt['val_macro_f1']:.4f}, val_loss={ckpt['val_loss']:.4f})")

    all_preds, all_labels = [], []
    with torch.no_grad():
        for imgs, labels in test_loader:
            imgs = imgs.to(DEVICE)
            logits = model_cnn(imgs)
            all_preds.extend(logits.argmax(1).cpu().numpy())
            all_labels.extend(labels.numpy())

    all_preds = np.array(all_preds); all_labels = np.array(all_labels)

    cnn_acc = accuracy_score(all_labels, all_preds)
    cnn_prec = precision_score(all_labels, all_preds, average="macro", zero_division=0)
    cnn_rec  = recall_score(all_labels, all_preds, average="macro", zero_division=0)
    cnn_f1_macro = f1_score(all_labels, all_preds, average="macro", zero_division=0)
    cnn_f1_weighted = f1_score(all_labels, all_preds, average="weighted", zero_division=0)

    print(classification_report(all_labels, all_preds, target_names=label_encoder.classes_, zero_division=0))
    cm = confusion_matrix(all_labels, all_preds, labels=range(len(AGE_CLASSES)))
    fig, ax = plt.subplots(figsize=(4, 4))
    ConfusionMatrixDisplay(cm, display_labels=label_encoder.classes_).plot(ax=ax, colorbar=False)
    ax.set_title("2D CNN — Mel image (test set)")
    plt.tight_layout(); plt.show()
```

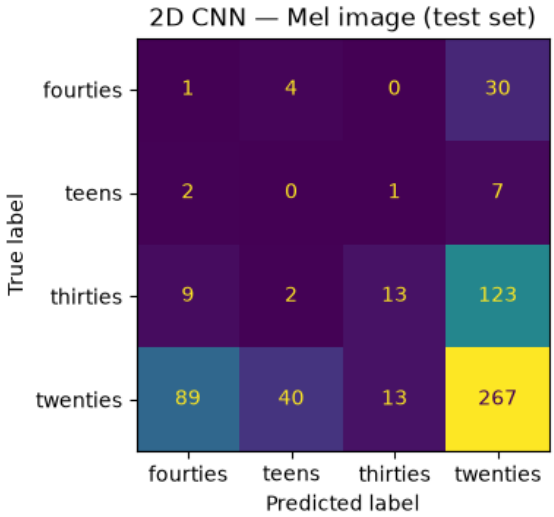
```python
    cnn_results_df = pd.DataFrame([{
        "Feature Set": "Mel image", "Model": "2D CNN",
        "Accuracy": cnn_acc, "Macro Precision": cnn_prec, "Macro Recall": cnn_rec,
        "Macro F1": cnn_f1_macro, "Weighted F1": cnn_f1_weighted,
    }])
    cnn_results_df.to_csv(RESULTS_DIR / "cnn_results.csv", index=False)
    cnn_results_df
else:
    cnn_results_df = pd.DataFrame()
    print("torch not installed -> CNN section skipped, cnn_results.csv not produced.")
```

```
Loaded best checkpoint from epoch 7 (val_macro_f1=0.3544, val_loss=1.3771)
              precision    recall  f1-score   support

    fourties       0.01      0.03      0.01        35
       teens       0.00      0.00      0.00        10
    thirties       0.48      0.09      0.15       147
    twenties       0.63      0.65      0.64       409

    accuracy                           0.47       601
   macro avg       0.28      0.19      0.20       601
weighted avg       0.54      0.47      0.47       601
```

- **CNN Test Results:** The best checkpoint achieved **47% test accuracy and 0.20 Macro F1**, with better classification of **twenties (F1 = 0.64)** but weak performance on teens, forties, and thirties.



- **CNN Confusion Matrix:** The model correctly classified most **twenties (267/409)** and **thirties (123/147)**, but frequently misclassified **forties and teens as twenties**, reflecting the class imbalance and limited minority-class recognition.

Final experiment comparison — all 10 experiments

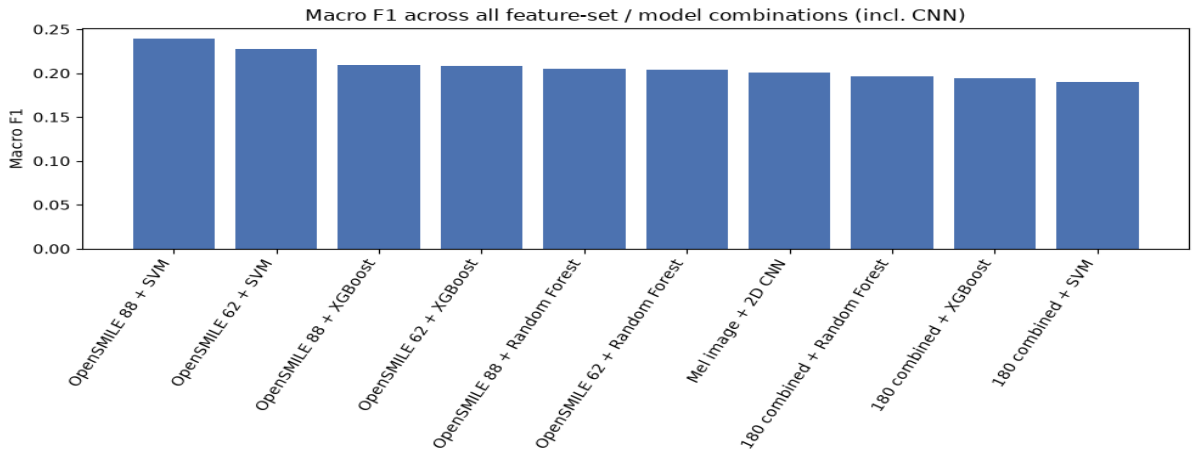
```
# ============================================================
# 24. FINAL COMPARISON TABLE
# ============================================================
final_cols = ["Feature Set", "Model", "Accuracy", "Macro Precision", "Macro Recall", "Macro F1", "Weighted F1"]
final_comparison_df = pd.concat(
    [ml_results_df[final_cols],
     cnn_results_df[final_cols] if len(cnn_results_df) else pd.DataFrame(columns=final_cols)],
    ignore_index=True
).sort_values("Macro F1", ascending=False).reset_index(drop=True)

final_comparison_df.to_csv(RESULTS_DIR / "final_comparison.csv", index=False)
print("=== FINAL COMPARISON (sorted by Macro F1) ===")
final_comparison_df
```

```
=== FINAL COMPARISON (sorted by Macro F1) ===
```

	Feature Set	Model	Accuracy	Macro Precision	Macro Recall	Macro F1	Weighted F1
0	OpenSMILE 88	SVM	0.627288	0.258439	0.251139	0.238913	0.567934
1	OpenSMILE 62	SVM	0.595674	0.240356	0.239521	0.227477	0.542830
2	OpenSMILE 88	XGBoost	0.584027	0.207406	0.226526	0.208943	0.522931
3	OpenSMILE 62	XGBoost	0.577371	0.213305	0.225171	0.208715	0.515531
4	OpenSMILE 88	Random Forest	0.653910	0.190833	0.242399	0.204527	0.546551
5	OpenSMILE 62	Random Forest	0.650582	0.191460	0.241176	0.203948	0.544914
6	Mel image	2D CNN	0.467554	0.279169	0.192455	0.200722	0.472099
7	180 combined	Random Forest	0.560732	0.183721	0.212527	0.196362	0.513845
8	180 combined	XGBoost	0.517471	0.187212	0.200992	0.193797	0.497268
9	180 combined	SVM	0.524126	0.180853	0.200169	0.189851	0.494765

- **Final Model Comparison: OpenSMILE 88 + SVM** achieved the highest **Macro F1 (0.239)**, while **OpenSMILE 88 + Random Forest** achieved the highest **accuracy (65.39%)**; overall, minority age-class performance remains limited.




```
======================================================================
FINAL REPORT
======================================================================

Dataset:
  Number of speakers: 142
  Number of clips: 7426
  Number of classes: 4  ['teens', 'twenties', 'thirties', 'fourties']

Speaker-independent split (method: v2-stratifiedgroupkfold7 ):
  Train speakers: 76  (6301 clips)
  Validation speakers: 18  (524 clips)
  Test speakers: 48  (601 clips)

Best measured ML results (by Macro F1):
  Feature Set         Model  Accuracy  Macro Precision  Macro Recall  Macro F1  Weighted F1
OpenSMILE 88           SVM  0.627288         0.258439      0.251139  0.238913     0.567934
OpenSMILE 62           SVM  0.595674         0.240356      0.239521  0.227477     0.542830
OpenSMILE 88       XGBoost  0.584027         0.207406      0.226526  0.208943     0.522931
OpenSMILE 62       XGBoost  0.577371         0.213305      0.225171  0.208715     0.515531
OpenSMILE 88 Random Forest  0.653910         0.190833      0.242399  0.204527     0.546551
OpenSMILE 62 Random Forest  0.650582         0.191460      0.241176  0.203948     0.544914
180 combined Random Forest  0.560732         0.183721      0.212527  0.196362     0.513845
180 combined       XGBoost  0.517471         0.187212      0.200992  0.193797     0.497268
180 combined           SVM  0.524126         0.180853      0.200169  0.189851     0.494765

2D CNN results:
Feature Set  Model  Accuracy  Macro Precision  Macro Recall  Macro F1  Weighted F1
  Mel image 2D CNN  0.467554         0.279169      0.192455  0.200722     0.472099

Final comparison (all experiments, sorted by Macro F1):
  Feature Set         Model  Accuracy  Macro Precision  Macro Recall  Macro F1  Weighted F1
OpenSMILE 88           SVM  0.627288         0.258439      0.251139  0.238913     0.567934
OpenSMILE 62           SVM  0.595674         0.240356      0.239521  0.227477     0.542830
OpenSMILE 88       XGBoost  0.584027         0.207406      0.226526  0.208943     0.522931
OpenSMILE 62       XGBoost  0.577371         0.213305      0.225171  0.208715     0.515531
OpenSMILE 88 Random Forest  0.653910         0.190833      0.242399  0.204527     0.546551
OpenSMILE 62 Random Forest  0.650582         0.191460      0.241176  0.203948     0.544914
    Mel image        2D CNN  0.467554         0.279169      0.192455  0.200722     0.472099
180 combined Random Forest  0.560732         0.183721      0.212527  0.196362     0.513845
180 combined       XGBoost  0.517471         0.187212      0.200992  0.193797     0.497268
180 combined           SVM  0.524126         0.180853      0.200169  0.189851     0.494765
```

- **Final Report Summary:** The dataset contains **7,426 clips from 142 speakers** across four age classes, using a speaker-independent split; **OpenSMILE 88 + SVM** achieved the highest Macro F1 (**0.239**), while **OpenSMILE 88 + Random Forest** achieved the highest accuracy (**65.39%**).

Question 3

Fine-tuning HuBERT and Wav2Vec2-XLS-R-53 for Indian Language Speech Recognition. Dataset Selection Criteria:

• Select a publicly available Indian language datasets

• Ensure that the dataset provide sufficient labeled audio data for fine-tuning.

Instructions:

- Implement CTC Loss for model training.
- Train both models and evaluate their performance using Word Error Rate (WER) and Character Error Rate (CER).

```python
import librosa
import numpy as np
import pandas as pd
import soundfile as sf
import torch
from transformers import AutoModelForCTC, get_linear_schedule_with_warmup

# ============================== CONFIG ==============================
DATA_ROOT = r"E:\sem 7\Speech Processing\assign 3"          # searched recursively for train.tsv
OUTPUT_DIR = r"E:\sem 7\Speech Processing\assign 3\outputs"  # fine-tuned checkpoints go here
RESULTS_CSV = "results.csv"
N_TRAIN, N_VAL, N_TEST = 2000, 100, 300
EPOCHS = 8
GRAD_ACCUM = 8
LR = 5e-5          # was 1e-5
HEAD_LR = 1e-3     # keep

MIN_SEC, MAX_SEC = 1.0, 8.0             # skip very short / long clips (saves VRAM)

BATCH_SIZE = 1                          # loop is written for batch size 1 (no padding needed)
WARMUP_FRAC = 0.1
SEED = 42
SAVE_MODELS = True

# (name, HF checkpoint, normalize waveform?, number of top encoder layers to train)
#  - HuBERT-base has 12 layers  -> train all of them
#  - XLS-R-53 has 24 layers     -> train only the top 6 (4 GB VRAM)
#  In both cases the CNN feature encoder is frozen; the CTC head is always trained.
MODELS = [
    ("HuBERT", "facebook/hubert-base-ls960", False, 12),
    ("Wav2Vec2-XLS-R-53", "facebook/wav2vec2-large-xlsr-53", True, 6),
]
# ====================================================================

SR = 16000
PAD, UNK, WORD_DELIM = "[PAD]", "[UNK]", "|"
PAD_ID = 0  # also used as the CTC blank
CTC = torch.nn.CTCLoss(blank=PAD_ID, reduction="mean", zero_infinity=True)
```

```python
# ------------------------------ text --------------------------------
def normalize_text(s: str) -> str:
    s = unicodedata.normalize("NFC", str(s))
    # drop punctuation (P*), symbols (S*) and zero-width format chars (Cf); keep Hindi letters/matras
    s = "".join(c for c in s if not unicodedata.category(c).startswith(("P", "S", "Cf")))
    return " ".join(s.split())


from collections import Counter

def build_vocab(texts, min_count=5):
    cnt = Counter(c for t in texts for c in t if c != " ")
    chars = sorted(c for c, n in cnt.items() if n >= min_count)
    tokens = [PAD, UNK, WORD_DELIM] + chars
    return {t: i for i, t in enumerate(tokens)}


def encode(text, vocab):
    return [vocab.get(WORD_DELIM if c == " " else c, vocab[UNK]) for c in text]


def decode(ids, id2tok):
    out, prev = [], None
    for i in ids:                          # greedy CTC decoding: collapse repeats, drop blank
        if i != prev and i != PAD_ID:
            out.append(id2tok[i])
        prev = i
    text = "".join(t for t in out if t != UNK).replace(WORD_DELIM, " ")
    return " ".join(text.split())

```

```python
# ------------------------------ data --------------------------------
def find_dataset(root):
    root = Path(root)
    if not root.exists():
        raise FileNotFoundError(f"DATA_ROOT does not exist: {root}")
    hits = sorted(root.rglob("train.tsv"))
    if not hits:
        raise FileNotFoundError(f"No train.tsv found under {root}. Extract the .tar.gz first.")
    lang_dir = hits[0].parent
    clips = lang_dir / "clips"
    if not clips.is_dir():
        alt = sorted(root.rglob("clips"))
        if not alt:
            raise FileNotFoundError(f"No 'clips' folder found under {root}")
        clips = alt[0]
    for s in ("train", "dev", "test"):
        if not (lang_dir / f"{s}.tsv").is_file():
            raise FileNotFoundError(f"Missing {lang_dir / (s + '.tsv')}")
    return lang_dir, clips


def read_tsv(tsv):
    # Common Voice TSVs contain stray quotes -> quoting=3 (QUOTE_NONE)
    df = pd.read_csv(tsv, sep="\t", quoting=3, dtype=str, usecols=["path", "sentence"]).dropna()
    df["text"] = df["sentence"].map(normalize_text)
    df = df[df["text"].str.len() > 0]
    return df.sample(frac=1.0, random_state=SEED).reset_index(drop=True)


def load_audio(path):
    wav, sr = sf.read(str(path), dtype="float32", always_2d=True)  # MP3 supported by soundfile>=0.12
    wav = wav.mean(axis=1)                                         # mono
    if sr != SR:
        wav = librosa.resample(wav, orig_sr=sr, target_sr=SR)      # 16 kHz
    return wav.astype(np.float32)

```

```python
def build_subset(df, clips, n, split):
    data, skipped, last_err = [], 0, None
    for path, text in zip(df["path"], df["text"]):
        if len(data) >= n:
            break
        f = clips / path
        if not f.is_file():
            skipped += 1
            continue
        try:
            wav = load_audio(f)
        except Exception as e:  # corrupt / undecodable mp3
            skipped += 1
            last_err = e
            continue
        if not (MIN_SEC <= len(wav) / SR <= MAX_SEC):
            skipped += 1
            continue
        data.append((wav, text))
    print(f"[data] {split}: kept {len(data)} clips (skipped {skipped})")
    if not data:
        raise RuntimeError(f"No usable audio for '{split}'. Last error: {last_err}. "
                           "If MP3 decoding failed, run: pip install -U soundfile (or install ffmpeg).")
    return data


def to_input(wav, normalize, device):
    x = torch.from_numpy(wav)
    if normalize:  # zero-mean / unit-variance (XLS-R expects this, HuBERT-base does not)
        x = (x - x.mean()) / torch.sqrt(x.var() + 1e-7)
    return x.unsqueeze(0).to(device)


# ------------------------------ model --------------------------------
def ctc_loss_fn(logits, ids):
    """CTC LOSS: logits (1,T,V) from the linear CTC head -> log-probs (T,1,V) -> nn.CTCLoss."""
    log_probs = logits.float().log_softmax(-1).transpose(0, 1)
    targets = torch.tensor([ids], dtype=torch.long, device=logits.device)
    return CTC(log_probs, targets, (logits.size(1),), (len(ids),))

```

```python
def set_trainable(model, top_k):
    base = model.base_model                     # model.hubert / model.wav2vec2
    layers = base.encoder.layers
    if top_k >= len(layers):
        for p in model.parameters():
            p.requires_grad = True
    else:
        for p in model.parameters():
            p.requires_grad = False
        for layer in layers[-top_k:]:
            for p in layer.parameters():
                p.requires_grad = True
        for p in base.encoder.layer_norm.parameters():
            p.requires_grad = True
        for p in model.lm_head.parameters():    # linear CTC head
            p.requires_grad = True
    model.freeze_feature_encoder()              # CNN feature extractor always frozen


# ---------------------------- metrics ----------------------------
def edit_distance(a, b):
    prev = list(range(len(b) + 1))
    for i, x in enumerate(a, 1):
        cur = [i]
        for j, y in enumerate(b, 1):
            cur.append(min(prev[j] + 1, cur[j - 1] + 1, prev[j - 1] + (x != y)))
        prev = cur
    return prev[-1]


def error_rate(refs, hyps, unit):
    errs = total = 0
    for r, h in zip(refs, hyps):
        r_, h_ = (r.split(), h.split()) if unit == "word" else (list(r), list(h))
        errs += edit_distance(r_, h_)
        total += len(r_)
    return 100.0 * errs / max(total, 1)

```

```python
# -------------------------- train / evaluate ----------------------
@torch.no_grad()
def predict(model, data, normalize, device, use_amp, id2tok):
    model.eval()
    refs, hyps = [], []
    for wav, text in data:
        with torch.autocast(device_type=device.type, dtype=torch.float16, enabled=use_amp):
            logits = model(input_values=to_input(wav, normalize, device)).logits
        hyps.append(decode(logits.argmax(-1)[0].cpu().tolist(), id2tok))
        refs.append(text)
        del logits
    return refs, hyps

@torch.no_grad()
def val_loss(model, data, vocab, normalize, device, use_amp):
    model.eval()
    tot = 0.0
    for wav, text in data:
        with torch.autocast(device_type=device.type, dtype=torch.float16, enabled=use_amp):
            logits = model(input_values=to_input(wav, normalize, device)).logits
        tot += ctc_loss_fn(logits, encode(text, vocab)).item()
    return tot / len(data)

def run_model(name, hf_id, normalize, top_k, vocab, train, val, test, device):
    print(f"\n{'=' * 60}\nFine-tuning {name}  ({hf_id})\n{'=' * 60}")
    use_amp = device.type == "cuda"
    id2tok = {i: t for t, i in vocab.items()}

    # Encoder + freshly initialised Linear CTC head (lm_head) sized to our Hindi vocab
    model = AutoModelForCTC.from_pretrained(
        hf_id, vocab_size=len(vocab), pad_token_id=PAD_ID, ctc_zero_infinity=True,
        layerdrop=0.0, mask_time_prob=0.05, ignore_mismatched_sizes=True,
    ).to(device)
    set_trainable(model, top_k)
    model.gradient_checkpointing_enable(gradient_checkpointing_kwargs={"use_reentrant": False})
    n_tr = sum(p.numel() for p in model.parameters() if p.requires_grad)
    n_all = sum(p.numel() for p in model.parameters())
    print(f"[model] trainable params: {n_tr / 1e6:.1f}M / {n_all / 1e6:.1f}M "
          f"(top {top_k} encoder layers + CTC head; CNN feature encoder frozen)")
```

```
    head = [p for n, p in model.named_parameters() if p.requires_grad and n.startswith("lm_head")]
    enc = [p for n, p in model.named_parameters() if p.requires_grad and not n.startswith("lm_head")]
    opt = torch.optim.AdamW([{"params": enc, "lr": LR}, {"params": head, "lr": HEAD_LR}], weight_decay=0.0)
    total_updates = max(1, math.ceil(len(train) * EPOCHS / GRAD_ACCUM))
    sched = get_linear_schedule_with_warmup(opt, int(WARMUP_FRAC * total_updates), total_updates)
    scaler = torch.amp.GradScaler(device.type, enabled=use_amp)

    t0 = time.time()
    for epoch in range(1, EPOCHS + 1):
        model.train()
        order = list(range(len(train)))
        random.shuffle(order)
        run_loss, n_seen, pending, oom = 0.0, 0, 0, 0
        for i, idx in enumerate(order, 1):
            wav, text = train[idx]
            ids = encode(text, vocab)
            try:
                with torch.autocast(device_type=device.type, dtype=torch.float16, enabled=use_amp):
                    logits = model(input_values=to_input(wav, normalize, device)).logits  # encoder + CTC head
                loss = ctc_loss_fn(logits, ids)                                           # <-- CTC LOSS
                if torch.isfinite(loss):
                    scaler.scale(loss / GRAD_ACCUM).backward()                            # <-- backprop
                    run_loss += loss.item()
                    n_seen += 1
                    pending += 1
                del logits, loss
            except torch.cuda.OutOfMemoryError:
                oom += 1
                torch.cuda.empty_cache()
            if (i % GRAD_ACCUM == 0 or i == len(order)) and pending > 0:
                scaler.unscale_(opt)
                torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
                scaler.step(opt)
                scaler.update()
                sched.step()
                opt.zero_grad(set_to_none=True)
                pending = 0
            if i % 50 == 0 or i == len(order):
                print(f"[train] epoch {epoch}/{EPOCHS}  sample {i}/{len(order)}  "
                      f"avg CTC loss {run_loss / max(n_seen, 1):.3f}  ({time.time() - t0:.0f}s)"
                      + (f"  OOM-skipped {oom}" if oom else ""))
                run_loss, n_seen = 0.0, 0
```

```
        print(f"[val] epoch {epoch}  CTC loss {val_loss(model, val, vocab, normalize, device, use_amp):.3f}")

    if SAVE_MODELS:
        out = Path(OUTPUT_DIR) / name
        out.mkdir(parents=True, exist_ok=True)
        model.save_pretrained(out)
        (out / "vocab.json").write_text(json.dumps(vocab, ensure_ascii=False, indent=1), encoding="utf-8")
        print(f"[save] checkpoint -> {out}")

    refs, hyps = predict(model, test, normalize, device, use_amp, id2tok)
    wer, cer = error_rate(refs, hyps, "word"), error_rate(refs, hyps, "char")
    print(f"\n{'=' * 30}\n{name} RESULTS\n{'=' * 30}\nWER: {wer:.2f}\nCER: {cer:.2f}\n")
    for r, h in list(zip(refs, hyps))[:5]:
        print(f"Reference:  {r}\nPrediction: {h}\n")
    return {"Model": name, "WER": round(wer, 2), "CER": round(cer, 2)}
```

```python
def main():
    random.seed(SEED)
    np.random.seed(SEED)
    torch.manual_seed(SEED)

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"[env] torch {torch.__version__} | torch CUDA build: {torch.version.cuda} | device: {device}")
    if device.type == "cuda":
        p = torch.cuda.get_device_properties(0)
        print(f"[env] GPU: {p.name} ({p.total_memory / 1024 ** 3:.1f} GB) | mixed precision (fp16): ON")
    else:
        print("[env] CUDA not available -> running on CPU (slow). Install a CUDA build of PyTorch.")

    lang_dir, clips = find_dataset(DATA_ROOT)
    print(f"[data] found dataset: {lang_dir}\n[data] clips folder: {clips}")

    train_df, val_df, test_df = (read_tsv(lang_dir / f"{s}.tsv") for s in ("train", "dev", "test"))
    print(f"[data] full split sizes  train={len(train_df)}  dev={len(val_df)}  test={len(test_df)}")
    vocab = build_vocab(train_df["text"])
    print(f"[data] vocab size: {len(vocab)} (incl. {PAD}, {UNK}, '{WORD_DELIM}')")

    train = build_subset(train_df, clips, N_TRAIN, "train")
    val = build_subset(val_df, clips, N_VAL, "val")
    test = build_subset(test_df, clips, N_TEST, "test")
    print(f"[data] subset sizes  train={len(train)}  val={len(val)}  test={len(test)}")

    results = []
    for name, hf_id, normalize, top_k in MODELS:
        results.append(run_model(name, hf_id, normalize, top_k, vocab, train, val, test, device))
        gc.collect()                    # model is out of scope -> free it before loading the next
        if device.type == "cuda":
            torch.cuda.empty_cache()

    print(f"\n{'Model':<22}{'WER':>8}{'CER':>10}\n{'-' * 40}")
    for r in results:
        print(f"{r['Model']:<22}{r['WER']:>8.2f}{r['CER']:>10.2f}")
    pd.DataFrame(results).to_csv(RESULTS_CSV, index=False)
    print(f"\nSaved {RESULTS_CSV}")


if __name__ == "__main__":
    main()
```

Output:

```
(.venv) PS E:\sem 7\Speech Processing\assign 3> python train_asr.py
[env] torch 2.11.0+cu128 | torch CUDA build: 12.8 | device: cuda
[env] GPU: NVIDIA GeForce RTX 3050 Laptop GPU (4.0 GB) | mixed precision (fp16): ON
[data] found dataset: E:\sem 7\Speech Processing\assign 3\1781715680033-cv-corpus-26.0-2026-06-12-hi\cv-corpus-26.0-2026
-06-12\hi
[data] clips folder: E:\sem 7\Speech Processing\assign 3\1781715680033-cv-corpus-26.0-2026-06-12-hi\cv-corpus-26.0-2026-
06-12\hi\clips
[data] full split sizes  train=4896  dev=2814  test=3342
[data] vocab size: 104 (incl. [PAD], [UNK], '|')
[data] train: kept 2000 clips (skipped 8)
[data] val: kept 100 clips (skipped 1)
[data] test: kept 300 clips (skipped 14)
[data] subset sizes  train=2000  val=100  test=300
```

Figure 1. Experimental environment and Hindi Common Voice dataset preparation

```
============================================================
Fine-tuning HuBERT  (facebook/hubert-base-ls960)
============================================================
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and
faster downloads.
Loading weights: 100%|                                    | 211/211 [00:00<?, ?it/s]
[transformers] HubertForCTC LOAD REPORT from: facebook/hubert-base-ls960
Key            | Status  |
---------------+---------+-
lm_head.bias   | MISSING |
lm_head.weight | MISSING |

Notes:
- MISSING:      those params were newly initialized because missing from the checkpoint. Consider training on your downs
tream task.
[model] trainable params: 90.3M / 94.5M (top 12 encoder layers + CTC head; CNN feature encoder frozen)
[train] epoch 1/8  sample 50/2000  avg CTC loss 19.372  (6s)
[train] epoch 1/8  sample 100/2000  avg CTC loss 18.881  (10s)
[train] epoch 1/8  sample 150/2000  avg CTC loss 20.291  (14s)
[train] epoch 1/8  sample 200/2000  avg CTC loss 18.854  (18s)
[train] epoch 1/8  sample 250/2000  avg CTC loss 17.573  (22s)
[train] epoch 1/8  sample 300/2000  avg CTC loss 16.278  (26s)
[train] epoch 1/8  sample 350/2000  avg CTC loss 14.480  (30s)
[train] epoch 1/8  sample 400/2000  avg CTC loss 9.770  (34s)
[train] epoch 1/8  sample 450/2000  avg CTC loss 6.576  (37s)
[train] epoch 1/8  sample 500/2000  avg CTC loss 4.386  (41s)
[train] epoch 1/8  sample 550/2000  avg CTC loss 3.733  (45s)
[train] epoch 1/8  sample 600/2000  avg CTC loss 3.537  (48s)
[train] epoch 1/8  sample 650/2000  avg CTC loss 3.505  (52s)
[train] epoch 1/8  sample 700/2000  avg CTC loss 3.574  (56s)
[train] epoch 1/8  sample 750/2000  avg CTC loss 3.514  (59s)
[train] epoch 1/8  sample 800/2000  avg CTC loss 3.502  (63s)
[train] epoch 1/8  sample 850/2000  avg CTC loss 3.434  (67s)
```

Figure 2. HuBERT fine-tuning using CTC loss

```
[save] checkpoint -> E:\sem 7\Speech Processing\assign 3\outputs\HuBERT

==============================
HuBERT RESULTS
==============================
WER: 65.77
CER: 23.57

Reference:  वह छुपाछुपी कमरे से निकला कितनी ने भी उसको नहीं देखा
Prediction: वात छुपाच पे कमने से निकला कितनी नैवी उस्कोनी नेखा

Reference:  व्यापार से देश की प्रगति होती है
Prediction: वयापार फे देश की प्रगती होती है

Reference:  तुम्हारी साईकल मेरी साईकल जैसी है
Prediction: तुम्मारी साईकन मेरी साइकल जेासीहै

Reference:  अस्पताल में डॉक्टर ने तत्काल सेकरवाया महिला का इलाज मौत
Prediction: अस पतान में दॉक्टो ने तात्रिक सकिर य मा हीला काएलाज मौत

Reference:  दिल्ली के कितानों की मुआवजे के चे देंगे केजरीवाल
Prediction: दिल्ली के कितानो की मुआवुचरी के छै दिठो केज्रीवार
```

Figure 3. HuBERT ASR predictions compared with reference transcriptions

```
============================================================
Fine-tuning Wav2Vec2-XLS-R-53  (facebook/wav2vec2-large-xlsr-53)
============================================================
Loading weights: 100%|                    | 422/422 [00:00<00:00, 27218.57it/s]
[transformers] Wav2Vec2ForCTC LOAD REPORT from: facebook/wav2vec2-large-xlsr-53
[model] trainable params: 75.7M / 315.5M (top 6 encoder layers + CTC head; CNN feature encoder frozen)
[train] epoch 1/8  sample 50/2000  avg CTC loss 19.581  (5s)
[train] epoch 1/8  sample 100/2000  avg CTC loss 19.716  (10s)
[train] epoch 1/8  sample 150/2000  avg CTC loss 19.082  (15s)
[train] epoch 1/8  sample 200/2000  avg CTC loss 18.801  (20s)
[train] epoch 1/8  sample 250/2000  avg CTC loss 19.563  (25s)
[train] epoch 1/8  sample 300/2000  avg CTC loss 18.985  (29s)
[train] epoch 1/8  sample 350/2000  avg CTC loss 23.157  (34s)
[train] epoch 1/8  sample 400/2000  avg CTC loss 18.009  (38s)
[train] epoch 1/8  sample 450/2000  avg CTC loss 17.910  (43s)
[train] epoch 1/8  sample 500/2000  avg CTC loss 15.054  (48s)
[train] epoch 1/8  sample 550/2000  avg CTC loss 16.815  (53s)
[train] epoch 1/8  sample 600/2000  avg CTC loss 14.502  (58s)
[train] epoch 1/8  sample 650/2000  avg CTC loss 14.152  (62s)
[train] epoch 1/8  sample 700/2000  avg CTC loss 12.447  (67s)
[train] epoch 1/8  sample 750/2000  avg CTC loss 12.088  (72s)
[train] epoch 1/8  sample 800/2000  avg CTC loss 9.773  (77s)
[train] epoch 1/8  sample 850/2000  avg CTC loss 8.759  (82s)
[train] epoch 1/8  sample 900/2000  avg CTC loss 7.071  (88s)
[train] epoch 1/8  sample 950/2000  avg CTC loss 6.146  (94s)
[train] epoch 1/8  sample 1000/2000  avg CTC loss 4.177  (101s)
```

Figure 5. Wav2Vec2-XLS-R-53 fine-tuning using CTC loss

```
[save] checkpoint -> E:\sem 7\Speech Processing\assign 3\outputs\Wav2Vec2-XLS-R-53

==============================
Wav2Vec2-XLS-R-53 RESULTS
==============================
WER: 85.82
CER: 33.77

Reference:  वह छुपाछुपी कमरे से निकला किसी ने भी उसको नहीं देखा
Prediction: वात चुपाचुपि कामनैह नीकला कसी नै बीयस कोनी लेा

Reference:  व्यापार से देश की प्रगति होती है
Prediction: वयापार फे देश की प्रागुती हाती है

Reference:  तुम्हारी साईकल मेरी साईकल जैसी है
Prediction: कतुमारीसाई कन मेरीसाकनजेाी हय

Reference:  अस्पताल में डॉक्टर ने तत्काल से करवाया महिला का इलाज मौत
Prediction: अस्पतान मे ाक टनें तानत्रि क सेवेर फह म हीला का लाचमौत

Reference:  दिल्ली के कितनों को मुआवजे के चेक देंगे केजरीवाल
Prediction: दिली के केनानों को मुीावजे केशे क देगे केजरीवा
```

Figure 6. Wav2Vec2-XLS-R-53 evaluation and sample ASR predictions

Model	WER	CER
HuBERT	65.77	23.57
Wav2Vec2-XLS-R-53	85.82	33.77

Figure 7. Final comparison of HuBERT and Wav2Vec2-XLS-R-53

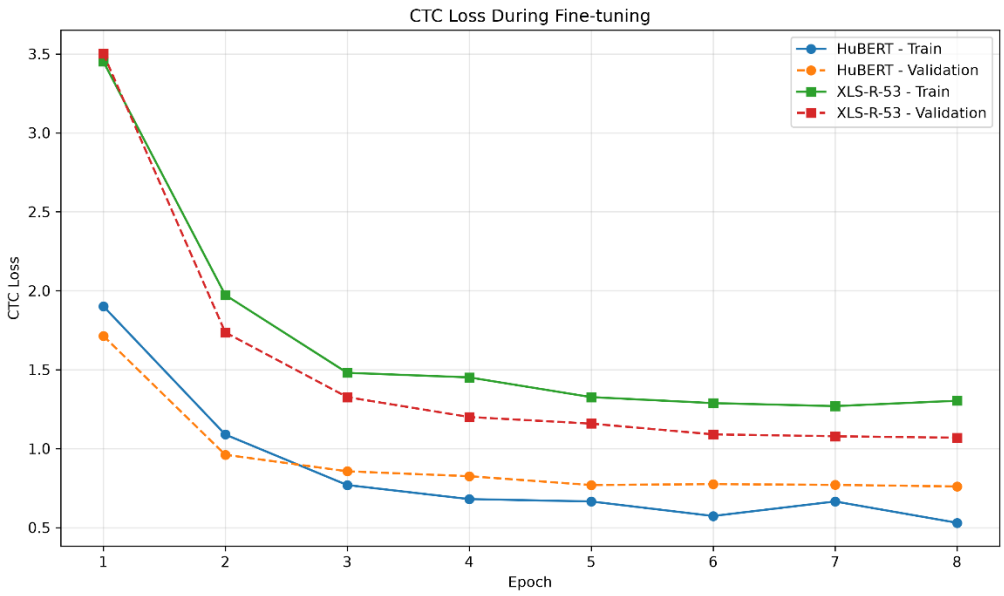
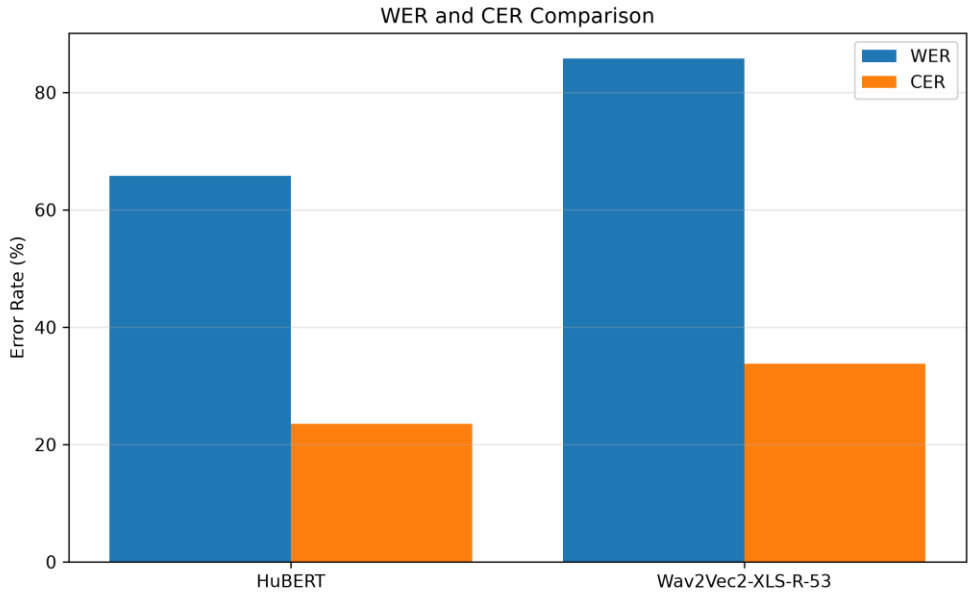


Figure: Training and Validation CTC Loss for HuBERT and Wav2Vec2-XLS-R-53

The CTC loss decreased progressively during fine-tuning for both models, indicating that the models learned to map the Hindi speech input to the character-level transcription targets. HuBERT showed a lower final validation CTC loss than Wav2Vec2-XLS-R-53 under the experimental configuration.



The WER and CER comparison shows that HuBERT achieved lower recognition error rates than Wav2Vec2-XLS-R-53 under the experimental conditions used. HuBERT obtained a WER of 65.77% and a CER of 23.57%, whereas Wav2Vec2-XLS-R-53 obtained a WER of 85.82% and a CER of 33.77%. This indicates that the fine-tuned HuBERT model produced fewer word-level and character-level transcription errors on the Hindi test subset. The results are specific to the selected Common Voice Hindi subset, vocabulary, training configuration, and

computational constraints, and therefore should not be interpreted as a general superiority of one model over the other.

Codes in GitHub: https://github.com/Harsha-AIE/speech-processing-age-classification-asr